



LAPORAN KERJA PRAKTIK- TF234703

ANALISIS PERANCANGAN PENGENDALI ROBUST H_2/H_∞ UNTUK SISTEM BELOK TERKOORDINASI MODE HALF-BANK PADA AUTO-PILOT PESAWAT CN235-220

Rama Suryansyah Budianto
NRP. 5009221003

Tempat Pelaksanaan Kerja Praktik
PT. Dirgantara Indonesia

Departemen Teknik Fisika
Fakultas Teknologi Industri dan Rekayasa Sistem
Institut Teknologi Sepuluh Nopember Surabaya
2020

LEMBAR PENGESAHAN

LAPORAN KERJA PRAKTIK

“ANALISIS PERANCANGAN PENGENDALI ROBUST H_2/H_∞ UNTUK SISTEM KOORDINASI BELOKAN MODE HALF-BANK PADA AUTO- PILOT PESAWAT CN235-220“

disusun oleh

Rama Suryansyah Budianto

NRP. 5009221003

Yang telah menyelesaikan mata kuliah TF-234703 Kerja Praktik di PT. Dirgantara
Indonesia, pada tanggal 7 Juli 2025 s/d 15 Agustus 2025

Bandung, 12 Agustus 2025

Menyetujui,

Dosen Pembimbing



Prof. Dr. Ir. Aulia Siti Aisjah, M.T.

NIP. 19660116 198903 2 001

Pembimbing Lapangan



Widi Handoko

NIK. 930369

Mengetahui,
Kepala Departemen Teknik Fisika



Prof. Gunawan Nugroho, S.T, M.T, Ph.D

NIP. 19771127200212 1 002

ANALISIS PERANCANGAN PENGENDALI ROBUST H_2/H_∞ UNTUK SISTEM BELOK TERKOORDINASI MODE HALF- BANK PADA AUTO-PILOT PESAWAT CN235-220

Nama Rama Suryansyah Budianto
NRP 5009221003
Departemen Teknik Fisika – FTIRS
Pembimbing 1. Prof. Dr. Ir. Aulia Siti Aisjah, M.T.
2. Widi Handoko

ABSTRAK

Manuver belok terkoordinasi pada mode *Half Bank* sistem autopilot pesawat CN235-220 memiliki potensi peningkatan dalam aspek kenyamanan penumpang dan efisiensi operasional. Studi ini bertujuan merancang sistem kendali *robust* menggunakan metode sintesis H_2/H_∞ untuk mengoptimalkan manuver tersebut dengan meminimalkan pergerakan lateral yang tidak diinginkan serta mengurangi beban kerja aktuator. Penulis menerapkan metodologi optimisasi hibrida yang menggabungkan *Linear Matrix Inequality* (LMI) untuk menjamin stabilitas sistem dengan algoritma *Quasi Newton* untuk mencari nilai *gain* kontroler optimal secara iteratif. Berdasarkan hasil simulasi, kontroler H_2/H_∞ campuran terbukti memberikan keseimbangan performa terbaik dibandingkan kontroler *Baseline*, H_2 murni, dan H_∞ murni. Implementasi kontroler ini berhasil menurunkan percepatan lateral RMS sebesar 11-12% yang berdampak positif pada kenyamanan, serta menurunkan *Rudder Rate* RMS hingga 25% yang mengindikasikan peningkatan efisiensi aktuator. Analisis data juga memperlihatkan adanya *trade off* terukur di mana sistem mengizinkan deviasi *sideslip* sesaat yang sedikit lebih besar demi menghasilkan manuver yang jauh lebih halus, sehingga memvalidasi efektivitas metode optimisasi hibrida untuk peningkatan sistem autopilot CN235-220.

.

Kata Kunci: Kontrol Robust, Sintesis H_2/H_∞ , Pesawat CN235-220, Belok terkoordinasi

KATA PENGANTAR

Puji syukur kepada Tuhan Yang Maha Esa karena atas izinnya, Penulis dapat menyelesaikan Laporan Kerja Praktik dengan judul “Analisis Perancangan Pengendali Robust H_2/H_∞ Untuk Sistem Belok terkoordinasi Mode Half-Bank Pada Auto-Pilot Pesawat CN235-220”. Penulisan laporan ini disusun sebagai salah satu syarat untuk menyelesaikan program studi Kerja Praktik di Departemen Teknik Fisika Institut Teknologi Sepuluh Nopember. Pelaksanaan kerja praktik ini memberikan banyak pengalaman berharga dan pengetahuan baru bagi Penulis. Dalam proses penyusunan laporan ini, penulis mendapat banyak bantuan dan bimbingan dari berbagai pihak. Oleh karena itu, penulis ingin mengucapkan terima kasih secara tulus kepada:

- a. Bapak Prof. Gunawan Nugroho, S.T., Ph.D. selaku Kepala Departemen Teknik Fisika Institut Teknologi Sepuluh Nopember.
- b. Ibu Prof. Dr. Ir. Aulia Siti Aisjah, M. T. selaku dosen wali sekaligus dosen pembimbing kampus yang selalu memberikan arahan dan bimbingan kepada penulis.
- c. Bapak Widi Handoko selaku pembimbing lapangan dari Departemen Rancang Bangun dan Analisis Sistem PT. Dirgantara Indonesia yang dengan sabar telah membimbing, memberikan kesempatan bagi penulis untuk berkembang dan memberikan ilmunya kepada penulis.
- d. Keluarga penulis yang senantiasa memberikan dukungan penuh kepada penulis.
- e. Rekan kerja praktik yang selalu memberikan dorongan semangat dan dukungan dalam bentuk materi ataupun moral kepada penulis.
- f. Serta pihak lain yang telah terlibat memberikan bimbingan dan bantuan baik secara langsung ataupun tidak langsung dalam pelaksanaan serta penyusunan laporan kerja praktik.

Dalam penulisan laporan ini, penulis menyadari bahwa materi yang telah dituangkan dalam laporan kerja praktik ini masih jauh dari kata sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran dari berbagai pihak untuk penyempurnaannya. Selanjutnya diharapkan laporan kerja praktik ini dapat bermanfaat bagi semua pihak yang membaca serta membutuhkan laporan ini sebagai referensi ataupun acuan dalam membuat media pembelajaran.

DAFTAR ISI

HALAMAN JUDUL.....	i
LEMBAR PENGESAHAN	iii
ABSTRAK	v
KATA PENGANTAR	vii
DAFTAR ISI.....	ix
DAFTAR GAMBAR	xi
DAFTAR TABEL.....	xiii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Ruang Lingkup	2
1.3 Profil Perusahaan PT Dirgantara Indonesia.....	2
1.4 Logo dan Makna Logo PT Dirgantara Indonesia	3
1.5 Struktur Organisasi	4
BAB II PELAKSANAAN KERJA PRAKTIK	7
2.1 Proses Pelaksanaan Kerja Praktik.....	7
2.2 Pesawat CN235-220	9
2.2.1 Bidang Kendali Pesawat CN235-220	10
2.3 Sistem <i>Autopilot</i> AP500R.....	12
2.3.1 Mode <i>Half Bank</i> Sistem <i>Autopilot</i> AP500R.....	14
2.4 Persamaan Gerak Lateral Pesawat.....	15
2.4.1 Definisi Vektor State dan Input	15
2.4.2 Matriks Dinamika Lateral Pesawat CN235-220.....	16
2.5 Kontrol <i>Robust</i>	16
2.5.1 Linear Matrix Inequality (LMI).....	18
2.5.2 Norma H_2	18
2.5.3 Norma H_∞	19
2.6 Analisis Pengendalian Sistem.....	20
2.7 Optimisasi Quasi Newton.....	20
2.7.1 Algoritma Quasi Newton BFGS.....	21
2.7.2 Batasan Kestabilan Kontrol <i>Robust</i>	21

BAB III HASIL PEMBELAJARAN	23
3.1 Deskripsi dan Tujuan Tugas Khusus	23
3.2 Metode Penyelesaian Tugas Khusus	24
3.3 Permodelan Belok Terkoordinasi	25
3.3.1 Kontroler <i>Yaw</i>	26
3.3.2 Kontroler <i>Sideslip</i>	28
3.3.3 Kontroler Rudder	29
3.3.4 Kontroler Aileron	29
3.3.5 Representasi Model Belok Terkoordinasi	30
3.4 Analisis Controllability dan Observability Permodelan	31
3.5 Analisis <i>Pole Placement</i> Permodelan	33
3.5.1 Analisis <i>Open Loop</i> Permodelan	34
3.5.2 Analisis <i>Closed Loop</i> Permodelan	35
3.6 Optimasi <i>Gain</i> Belok Terkoordinasi	35
3.6.1 Formulasi Fungsi Biaya	36
3.6.2 Batas Variabel Keputusan	36
3.7 Parameter Inisialisasi Simulasi	37
3.8 Pembahasan	39
3.9 Diskusi	41
BAB IV PENUTUP	43
4.1 Kesimpulan	43
4.2 Saran	44
DAFTAR PUSTAKA	45
LAMPIRAN	47
Dokumentasi Pelaksanaan Kerja Praktik	47
<i>Source Code</i> Program MATLAB	47
CN235-220 Flight Test Report	60

DAFTAR GAMBAR

Gambar 1.1 Logo PT. Dirgantara Indonesia	4
Gambar 1.2 Struktur Organisasi PT. Dirgantara Indonesia	5
Gambar 1.3 Struktur Organisasi Divisi Pusat Inovasi & Teknologi Kedirgantaraan	6
Gambar 2.1 Pesawat CN235-220	9
Gambar 2.2 <i>Primary Control Surface</i> pada Pesawat	11
Gambar 2.3 <i>Trim Tabs</i> pada Pesawat	11
Gambar 2.4 Arsitektur Sistem <i>Autopilot</i> CN235-220	13
Gambar 2.5 Lokasi Komponen Sistem <i>Autopilot</i> CN235-220	14
Gambar 2.6 Plant Loop Kontrol dengan <i>Uncertainty</i>	17
Gambar 2.7 Diagram Alir Optimasi Dinamika Lateral Pesawat	22
Gambar 3.1 Diagram Alir Metode Penyelesaian Tugas Khusus	24
Gambar 3.2 Blok Diagram Sistem Belok Terkoordinasi Keseluruhan	26
Gambar 3.3 Blok Diagram Loop Dalam (<i>Inner Loop</i>) Kontrol <i>Yaw</i>	26
Gambar 3.4 Blok Diagram Loop Luar (<i>Outer Loop</i>) Kontrol <i>Sideslip</i>	28
Gambar 3.5 Blok Diagram Kontroler Rudder.....	29
Gambar 3.6 Blok Diagram Kontroler Aileron	29
Gambar 3.7 <i>Pole Placement Open Loop</i> Permodelan.....	34
Gambar 3.8 <i>Pole Placement Closed Loop</i> Permodelan	35
Gambar 3.9 Grafik Performa Simulasi Dinamika lateral Pesawat.....	39
Gambar 3.10 Legenda Grafik Performa Simulasi Dinamika lateral Pesawat.....	39

DAFTAR TABEL

Tabel 1.1 Unit Bisnis PT. Dirgantara Indonesia	3
Tabel 2.1 Proses Pelaksanaan Kerja Praktik	8
Tabel 2.2 Spesifikasi Teknis Pesawat CN235-220	9
Tabel 2.3 Mode Operasi pada Sistem <i>Autopilot</i> AP500R.....	12
Tabel 3.1 Parameter Konstanta dan Konversi Simulasi Mode <i>Half Bank</i>	37
Tabel 3.2 Parameter Kondisi Penerbangan Simulasi Mode <i>Half Bank</i>	37
Tabel 3.3 Parameter Gain Kontroler Dasar Simulasi Mode <i>Half Bank</i>	38
Tabel 3.4 Parameter Desain dan Batasan LMI Simulasi Mode <i>Half Bank</i>	38
Tabel 3.5 Parameter Sistem Simulasi Mode <i>Half Bank</i>	38
Tabel 3.6 Parameter Pengaturan Simulasi Mode <i>Half Bank</i>	38
Tabel 3.7 Matriks Performa Simulasi Dinamika lateral Pesawat	39

BAB I

PENDAHULUAN

1.1 Latar Belakang

Industri kedirgantaraan terus berkembang untuk meningkatkan keamanan, kenyamanan, dan efisiensi pengoperasian pesawat. PT Dirgantara Indonesia sebagai salah satu produsen pesawat di kawasan Asia juga melakukan peningkatan berkelanjutan pada produk-produknya, termasuk pesawat angkut multiguna CN235-220. Salah satu upaya peningkatan yang menjadi fokus adalah pengembangan performa sistem autopilot. Pengembangan ini penting dilakukan agar CN235-220 tetap bersaing pada pasar global dan dapat memenuhi kebutuhan operasi penerbangan modern yang semakin menuntut presisi dan keandalan.

Salah satu aspek penting dalam operasional pesawat adalah manuver belok. Khusus pada mode *half-bank* autopilot pesawat CN235-220 mengimplementasikan pembatasan sudut guling untuk meningkatkan kenyamanan penumpang. Tetapi, saat pesawat bermanuver interaksi antara kemudi guling (*aileron*) dan kemudi belok (*rudder*) dapat menyebabkan gerakan lateral yang tidak terkoordinasi yang dikenal sebagai *slip* atau *skid*. Fenomena *slip* dan *skid* ini dapat mengurangi kenyamanan dan meningkatkan beban kerja aktuator. Oleh karena itu, belok terkoordinasi harus dioptimalkan untuk menjaga keseimbangan aerodinamis *lateral* secara presisi.

Sistem dinamika *lateral* pesawat merupakan subjek yang kompleks dan sering dipengaruhi ketidakpastian (*uncertainty*) akibat variasi parameter penerbangan dan gangguan lingkungan seperti turbulensi angin. Perancangan sistem kendali pada kondisi yang tidak pasti ini memerlukan pendekatan yang *robust* untuk menjamin stabilitas dan performa sistem dalam skenario terburuk. Metode sintesis H_2/H_∞ merupakan satu metode kontrol yang memberikan sifat *robust*, sehingga sesuai dengan kebutuhan yang diinginkan. Metode H_2 bertujuan meminimalkan energi respons sistem agar mencapai efisiensi dan performa yang baik, sedangkan metode H_∞ menjamin ketahanan (*robustness*) terhadap ketidakpastian.

Tugas khusus kerja praktik diusulkan secara spesifik berfokus pada perancangan pengendali *state feedback* yang disintesis menggunakan optimisasi hibrida. Metodologi hibrida memanfaatkan *Linear Matrix Inequality* (LMI) untuk memastikan stabilitas dan ketahanan sistem, serta algoritma Quasi Newton untuk secara iteratif menemukan *gain* kontroler optimal yang meminimalkan fungsi biaya yang berorientasi pada performa penerbangan dan efisiensi aktuator. Dengan membandingkan kontroler H_2 murni, H_∞ murni, dan H_2/H_∞ campuran terhadap kontroler dasar (*baseline*), tugas khusus bertujuan menghasilkan rancangan sistem kendali untuk meningkatkan performa manuver belok terkoordinasi mode *half-bank* pada autopilot pesawat CN235-220.

1.2 Ruang Lingkup

Pelaksanaan kerja praktik berlokasi di PT Dirgantara Indonesia Divisi Pusat Inovasi & Teknologi Kedirgantaraan Departemen Rancang Bangun & Analisis Sistem – TD4000 dengan studi kasus adalah sistem kendali manuver belok secara terkoordinasi (*coordinated turn*), sebagai bagian dari kendali terbang sistem *autopilot* pesawat CN235-220.

1.3 Profil Perusahaan PT Dirgantara Indonesia

PT Dirgantara Indonesia atau Indonesian Aerospace (IAe) merupakan salah satu perusahaan kedirgantaraan terkemuka di Asia. Perusahaan ini memiliki kompetensi inti pada desain, pengembangan, pembuatan, dan perawatan wahana udara. Produk-produknya melayani kebutuhan sipil dan militer (PT Dirgantara Indonesia, n.d.).

PT Dirgantara Indonesia didirikan sejak tahun 1976. Perusahaan telah berhasil mengembangkan kemampuan sebagai industri manufaktur. Perusahaan memiliki diversifikasi produk di berbagai bidang, tidak hanya pada wahana udara, tetapi juga daerah lain. Bidang-bidang lain tersebut mencakup Teknologi Informasi, Otomotif, Maritim, Simulasi Teknologi, Industri Turbin, dan Rekayasa layanan (PT Dirgantara Indonesia, n.d.).

PT Dirgantara Indonesia telah menghasilkan lebih dari 300 unit pesawat dan helikopter, sistem pertahanan, komponen pesawat, serta layanan lainnya. Melalui

pelaksanaan program restrukturisasi pada awal tahun 2004, perusahaan kini didukung oleh 3.720 karyawan, dari 9.670 karyawan sebelumnya. Karyawan-karyawan tersebar di beberapa unit bisnis, unit bisnis tersebut dapat dilihat sebagai pada Tabel 1.1.

Tabel 1.1 Unit Bisnis PT. Dirgantara Indonesia
(sumber: PT Dirgantara Indonesia, n.d.)

Unit	Sub-unit
<i>Aircraft</i>	Pesawat
	Helikopter
<i>Aircraft Services</i>	<i>Maintenance</i>
	<i>Overhaul</i>
	Perbaikan dan Perubahan
<i>Aerostructure</i>	<i>Parts & Components</i>
	<i>Assemblies</i>
	<i>Assemblies Tools & Equipment</i>
<i>Engineering Services</i>	<i>Communication Technology</i>
	Simulator Teknologi
	<i>Information Technology Solution</i>
	<i>Design Center</i>

Industri wahana udara diharapkan menjadi lebih efisien dan produktif di masa depan. Industri juga perlu beradaptasi menjadi institusi bisnis yang berkelanjutan (PT Dirgantara Indonesia, n.d.).

PT Dirgantara Indonesia saat ini meliputi wilayah bangunan seluas 86,98 hektar. Kegiatan produksi perusahaan ditopang oleh 232 unit mesin dan peralatan yang beragam. Selain itu, beberapa peralatan lainnya tersebar di berbagai perakitan, laboratorium, dan unit pelayanan serta pemeliharaan (PT Dirgantara Indonesia, n.d.).

1.4 Logo dan Makna Logo PT Dirgantara Indonesia

Logo PT Dirgantara Indonesia ditunjukkan pada Gambar 1.1 sebagai berikut.



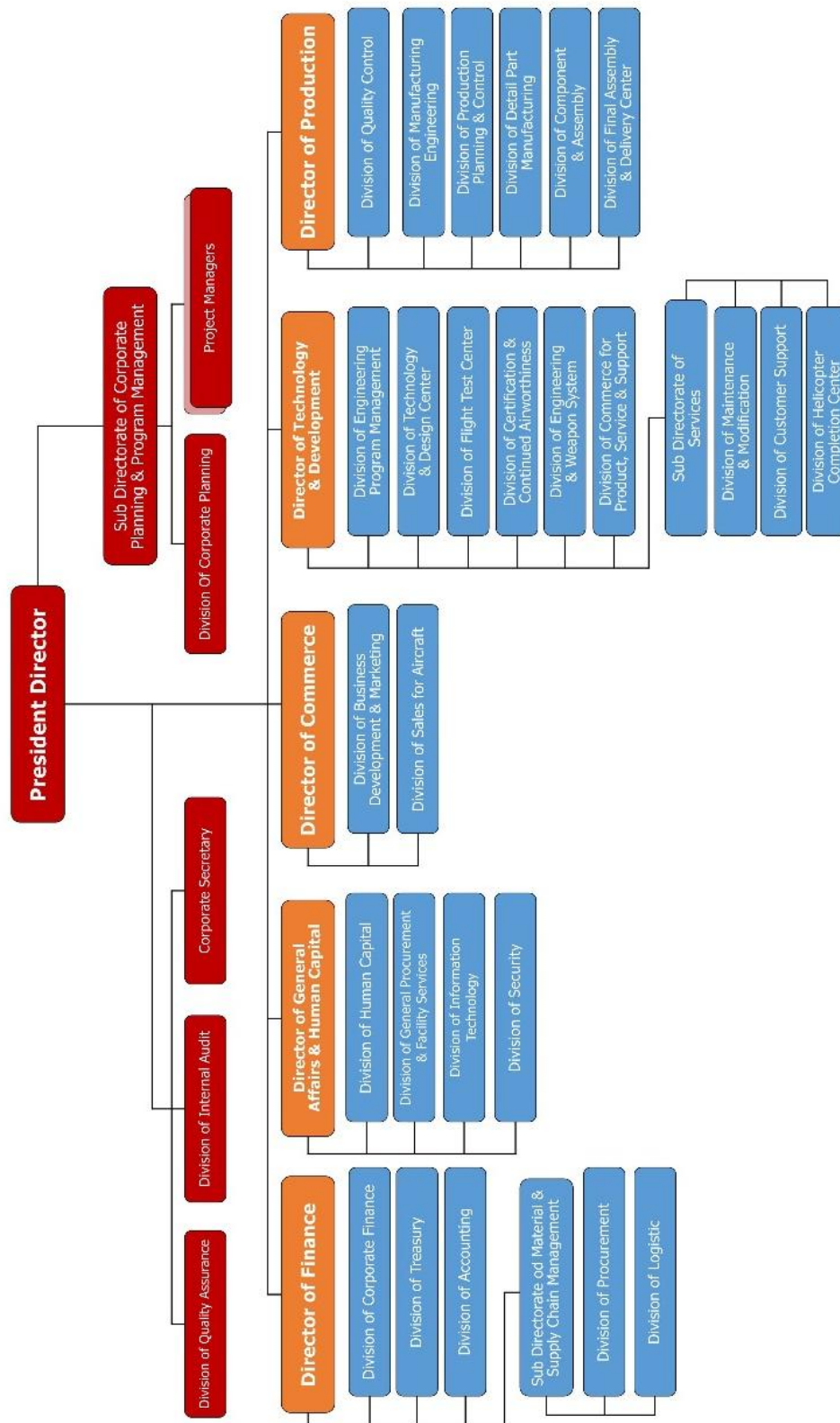
Gambar 1.1 Logo PT. Dirgantara Indonesia
(sumber: <https://www.indonesian-aerospace.com/en/>)

Makna pada logo PT. Dirgantara Indonesia adalah sebagai berikut.

- a. Warna biru angkasa melambangkan langit tempat pesawat terbang.
- b. Sayap pesawat terbang sebanyak 3 buah, melambangkan fase PT. Dirgantara Indonesia yaitu
 1. PT. Industri Pesawat Terbang Nurtanio
 2. PT. Industri Pesawat Terbang Nusantara
 3. PT. Dirgantara Indonesia
- c. Pada ukuran pesawat terbang yang semakin membesar melambangkan keinginan PT Dirgantara Indonesia untuk menjadi perusahaan dirgantara yang semakin membesar disetiap fasenya.
- d. Lingkaran melambangkan bola dunia di mana PT Dirgantara Indonesia ingin menjadi perusahaan kelas dunia.

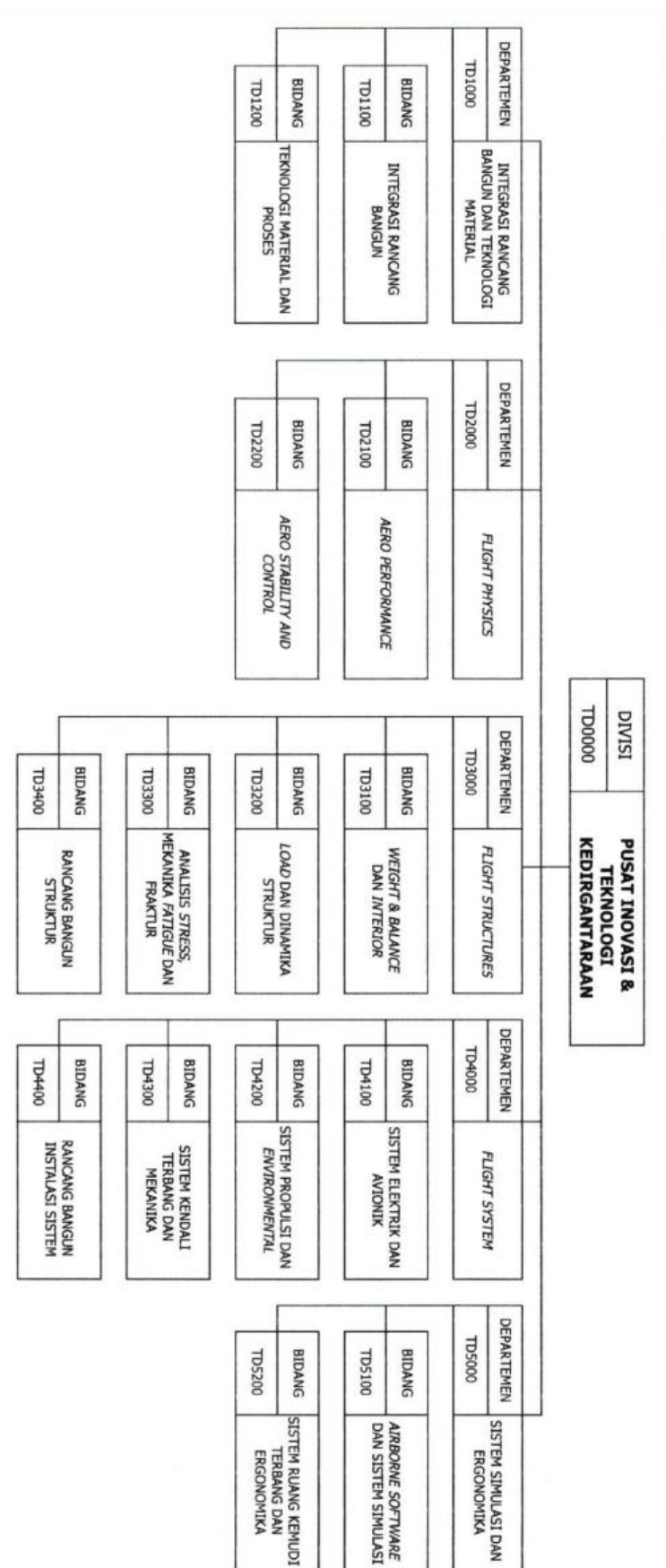
1.5 Struktur Organisasi

Bagan struktur organisasi PT Dirgantara Indonesia ditunjukkan pada Gambar 1.2 sebagai berikut.



Gambar 1.2 Struktur Organisasi PT. Dirgantara Indonesia
(sumber: <https://www.indonesian-aerospace.com/en/>)

Bagan struktur organisasi tingkat bidang lingkungan Divisi Pusat Inovasi & Teknologi Kedirgantaraan ditunjukkan pada Gambar 1.2 sebagai berikut.



Gambar 1.3 Struktur Organisasi Divisi Pusat Inovasi & Teknologi Kedirgantaraan
(sumber: <https://www.indonesian-aerospace.com/en/>)

BAB II

PELAKSANAAN KERJA PRAKTIK

2.1 Proses Pelaksanaan Kerja Praktik

Kerja praktik di PT Dirgantara Indonesia dilaksanakan selama 6 minggu, mulai tanggal 07 Juli 2025 hingga 15 Agustus 2025. Kegiatan berlangsung secara Work From Office (WFO) di Divisi Pusat Inovasi & Teknologi Kedirgantaraan, Departemen Rancang Bangun & Analisis Sistem – TD4000 pada bagian Flight Control System. Jadwal kerja praktik dimulai setiap hari Senin hingga Jumat, pukul 08.30 WIB dan berakhir pada pukul 15.30 WIB.

Kerja praktik memiliki objek studi yang dipilih adalah analisis sistem *autopilot Flight Guidance Computer AP500R* pada pesawat CN235-220. Tugas khusus yang diberikan berfokus pada perancangan dan pengembangan sistem kendali *state feedback* untuk mengoptimalkan manuver belok terkoordinasi (*coordinated turn*) pada mode *half bank* pesawat CN235-220. Rangkaian kegiatan yang dilakukan selama kerja praktik dijelaskan pada Tabel 2.1 sebagai berikut.

Tabel 2.1 Proses Pelaksanaan Kerja Praktik

No		Jenis Kegiatan	Waktu Pelaksanaan																																			
			Minggu 1						Minggu 2						Minggu 3						Minggu 4						Minggu 5						Minggu 6					
			Juli																														Agustus					
			7	8	9	10	11	14	15	16	17	18	21	22	23	24	25	28	29	30	31	1	4	5	6	7	8	11	12	13	14	15						
1	Registrasi dan administrasi																																					
2	Safety induction dan tata tertib pelaksanaan kerja praktik																																					
3	Pengenalalan profil perusahaan dan departemen																																					
4	Pengeralan objek studi kerja praktik																																					
5	Studi literatur objek studi dan tugas khusus																																					
6	Analisis data teknis objek studi																																					
7	Permodelan matematis dan perancangan sistem kendali																																					
8	Perancangan program simulasi MATLAB																																					
9	Asistensi pengerjaan tugas khusus																																					
10	Penyusunan laporan																																					
11	Administrasi dan evaluasi pembimbing lapangan																																					

2.2 Pesawat CN235-220

Pesawat CN235 adalah pesawat multiguna yang dirancang dengan kemampuan *Short Take-Off and Landing* (STOL), serta dilengkapi ramp door untuk memudahkan proses keluar dan masuk kargo. Pesawat ini juga dikenal memiliki biaya perawatan yang relatif rendah. CN235 mulai dikembangkan pada 17 Oktober 1979 melalui kerja sama antara Industri Pesawat Terbang Nusantara (IPTN, kini PT Dirgantara Indonesia) dan CASA (kini Airbus Defence & Space). Seiring perkembangan kemampuan PT Dirgantara Indonesia versi CN235 yang telah disempurnakan, seperti varian 110 dan 220. Setiap versi kemudian dikembangkan lagi menjadi sub-varian sesuai kebutuhan misi untuk sektor sipil, operator militer, maupun misi khusus (PT Dirgantara Indonesia, n.d.).



Gambar 2.1 Pesawat CN235-220
(sumber: <https://www.kkip.go.id/>)

CN235-220 termasuk dalam kelas pesawat angkut sedang (*medium transport aircraft*) wahana udara sayap tetap (*fixed wing*) dengan konfigurasi *high wing*. Pesawat ini menggunakan konstruksi *body semi monocoque* dan sistem penggerak berupa dua mesin turboprop (*twin engined turboprop*). Spesifikasi teknis lengkap pesawat CN235-220 dapat dilihat pada Tabel 2.2 sebagai berikut.

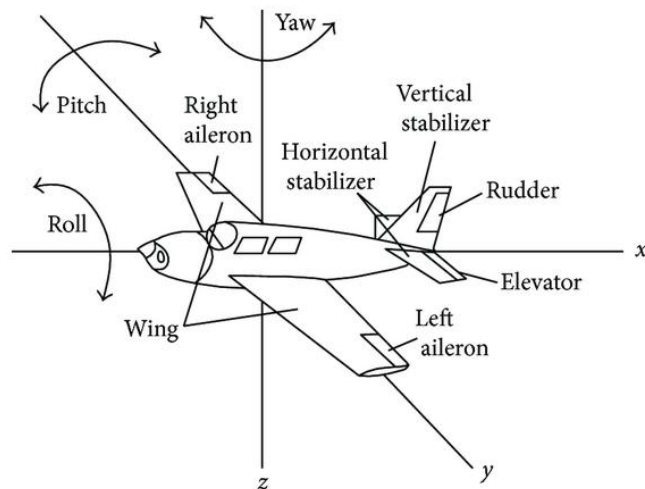
Tabel 2.2 Spesifikasi Teknis Pesawat CN235-220
(sumber: Technical Document CN235-220)

Overall Length	21.40 m	70 ft 2"
Overall Height	8.17 m	26 ft 10"
Cross Wing Area	59.1 m²	639 sq ft
Wing Span	25.81 m	84 ft 8"
Wing Aspect Ratio	10.156	

<i>Wheel Track</i>	3.9 m	12 ft 9"
<i>Wheel Base</i>	6.2 m	22 ft 4"
<i>Cabin Length (exc.ramp)</i>	9.65 m	31 ft 8"
<i>Ramp Length</i>	3.04 m	9 ft 11"
<i>Cargo hold Weight</i>	1.9 m	6 ft 3"
<i>Cargo Hold Max Width</i>	2.7 m	8 ft 10"
<i>Gross Cargo Hold Volume</i>	45.22 m ³	1597 cu ft
<i>Maximum Take Off Weight</i>	16,500 kg	36,380 lb
<i>Maximum Landing Weight</i>	16,500 kg	36,380 lb
<i>Maximum Zero Fuel Weight</i>	15,400 kg	33,950 lb
<i>Maximum Payload</i>	5,950 kg	13,120 lb
<i>Fuel Capacity</i>	5220 liters	1380 US galon
<i>Maximum Cruise Speed</i>	452 km/h	245 ktas
<i>Maximum Operating Altitude</i>	7620 m	25,000 ft
<i>Range with Maximum Fuel</i>	4132 km	2233 nm
<i>Ferry Range</i>	5055 km	2730 nm
<i>Take-off Run (ISA/SL)</i>	404 m	1325 ft
<i>Landing Roll (ISA/SL)</i>	378 m	1240 ft

2.2.1 Bidang Kendali Pesawat CN235-220

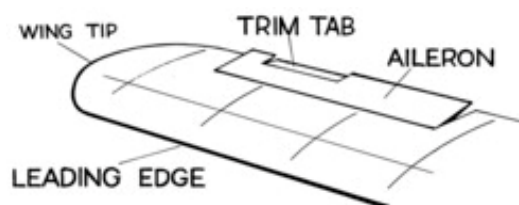
Pesawat CN235-220 memiliki bidang kendali yang dibagi menjadi dua bagian, yaitu *primary control surface* dan *secondary control surface*. *Primary control surface* berfungsi sebagai bidang kendali utama untuk mengatur arah gerak pesawat terhadap sumbu X (*roll*), Y (*pitch*), dan Z (*yaw*). Gambaran selengkapnya dapat dilihat pada Gambar 2.2 sebagai berikut.



Gambar 2.2 *Primary Control Surface* pada Pesawat
(sumber: <https://www.ilearnengineering.com>)

Bidang kendali ini terdiri dari tiga komponen, yaitu *aileron* yang terletak di kedua ujung sayap mengendalikan gerakan *roll* di sekitar sumbu Z, *elevator* yang berada pada *horizontal stabilizer* mengendalikan gerakan *pitch* di sekitar sumbu Y, dan *rudder* yang terpasang pada *vertical stabilizer* mengendalikan gerakan *yaw* di sekitar sumbu X. Posisi dan arah kerja masing-masing bidang kendali akan membantu menghasilkan respon gerak pesawat.

Secondary control surface berfungsi sebagai bidang kendali tambahan yang membantu meningkatkan kinerja penerbangan dengan memodifikasi karakteristik aerodinamika pesawat. Pada pesawat CN23-220 terdapat dua komponen pada *secondary control surface* terdiri dari *flaps* yang digunakan untuk meningkatkan gaya angkat dan menurunkan kecepatan saat lepas landas maupun mendarat, dan *trim tabs* yang berfungsi menjaga kestabilan kontrol selama penerbangan (PT Dirgantara Indonesia, n.d.).



Gambar 2.3 *Trim Tabs* pada Pesawat
(sumber: https://en.wikipedia.org/wiki/Flight_control_surfaces)

2.3 Sistem Autopilot AP500R

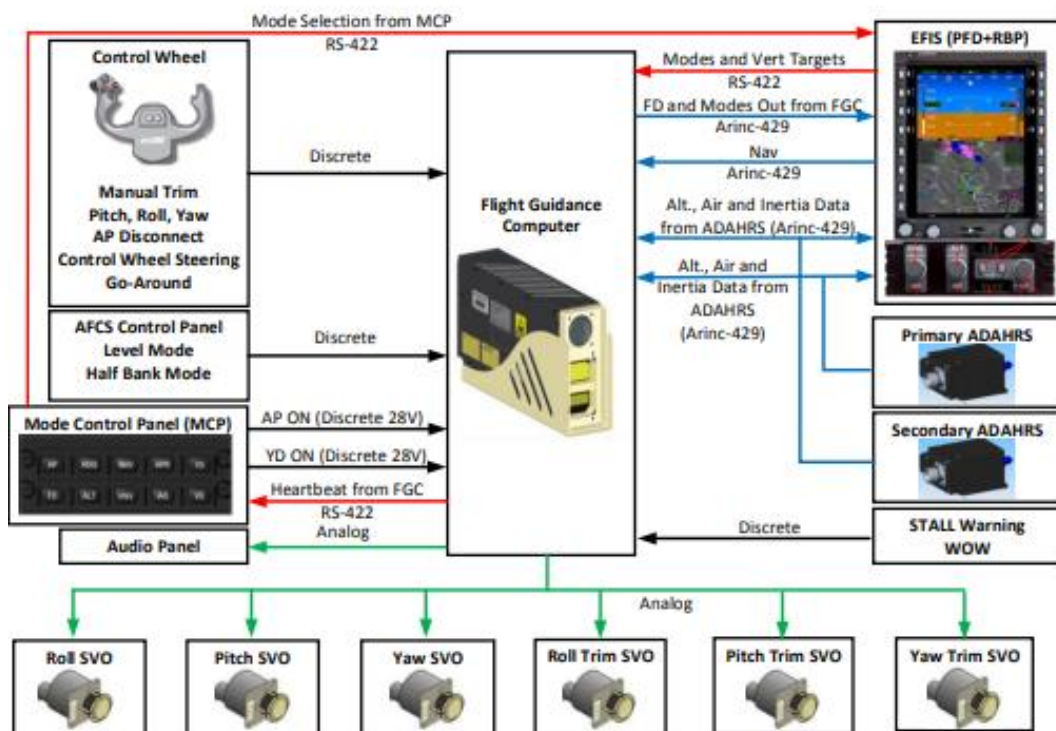
Flight Guidance Computer AP500R dari Genesys Aerosystems merupakan sistem autopilot tiga sumbu yang mampu mengendalikan gerakan pesawat pada sumbu X (*roll*), Y (*pitch*), dan Z (*yaw*). *Flight Guidance Computer* AP500R menyediakan berbagai mode operasi yang dapat dipilih oleh pilot selama penerbangan untuk membantu menjaga kestabilan dan memudahkan pengendalian pesawat (PT Dirgantara Indonesia, n.d.). Daftar lengkap mode operasi yang tersedia pada autopilot AP500R dapat dilihat pada Tabel 2.3 sebagai berikut.

Tabel 2.3 Mode Operasi pada Sistem *Autopilot* AP500R
(sumber: PTDI Autopilot AP500R for CN235-220)

Roll Channel	
ROLL	Provides commands to hold a target <i>roll</i> attitude (bank angle)
HDG	Provides commands to hold a target magnetic heading
NAV	Provides commands to hold a specified course (VOR,TACAN, etc)
APR	Provides commands to hold a specified approach course (ILS, GPS, etc)
BC	provides commands to hold a specified back course approach
Pitch Channel	
PITCH	Provides commands to hold a target <i>pitch</i> attitude
ALT Hold	Provides commands to hold a target altitude
ALT Preselect	Provides a means to capture a selected altitude from an active <i>pitch</i> mode
VS	Provides commands to hold a target vertical speed to a target altitude (if one provided) or hold a target vertical speed indefinitely
IAS	Provides commands to hold a target vertical speed to a target altitude (if one provided) or hold a target vertical speed indefinitely
GS	Provides commands to navigate vertically down the ILS glidepath
Yaw Channel	
YD	Provides damping in the <i>Yaw</i> axis and coordination of turns using the rudder.
Supplemental Modes	
CWS	Interrupt the servos in order to temporarily take manual control of the aircraft and/or re-sync the attitude/altitude/VS/IAS targets
GA	Disconnects the AP, engages FD to <i>ROLL</i> and <i>PITCH</i> mode, and sets the <i>roll</i> target to 0 deg and the <i>pitch</i> target to a configured nose up (typically 8deg) command
LVL	Engages <i>ROLL</i> and <i>PITCH</i> mode from and sets the <i>roll</i> target to 0 deg and the <i>pitch</i> target to 2 deg
HB	Sets the commanded <i>roll</i> target and <i>roll</i> command limit to half to produce a smoother ride.

TOGA	Disconnects the AP, engages FD to <i>ROLL</i> and <i>PITCH</i> mode, and sets the <i>roll</i> target to 0 deg and the <i>pitch</i> target to a configured nose up (typically 11 deg) command
------	--

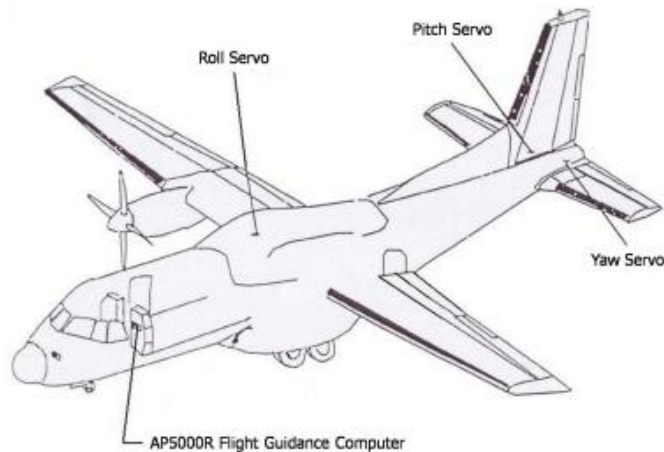
Sistem *autopilot* AP500R pengendalian tiap sumbu dilakukan dengan memanfaatkan data dari *Air Data, Attitude, and Heading Reference System* (ADAHRS) serta input dari sistem navigasi yang dipilih, seperti terlihat pada Gambar 2.4 sebagai berikut.



Gambar 2.4 Arsitektur Sistem *Autopilot* CN235-220
(sumber: PTDI Autopilot AP500R for CN235-220)

Sumbu *roll* autopilot membaca informasi berupa sikap *roll*, laju *roll*, kesalahan heading, dan kesalahan *course* dari ADAHRS, serta *deviasi course* dari penerima navigasi. Sumbu *pitch* autopilot membaca informasi berupa sikap *pitch*, laju *pitch*, ketinggian tekanan, kecepatan udara terindikasi, kecepatan *vertikal*, dan percepatan *vertikal* dari ADAHRS, ditambah *deviasi glideslope* dari penerima *glideslope*. Sumbu *yaw* autopilot membaca informasi berupa laju *yaw* dan *input* kecepatan dari ADAHRS (PT Dirgantara Indonesia, n.d.).

Data input menjadi umpan balik bagi *Flight Guidance Computer* (FGC) untuk menghitung perintah koreksi, sebagaimana diperlihatkan pada Gambar 2.5 sebagai berikut.



Gambar 2.5 Lokasi Komponen Sistem *Autopilot* CN235-220
(sumber: PTDI Autopilot AP500R for CN235-220)

FGC memberikan perintah menggerakkan *servo roll*, *servo pitch*, dan *servo yaw* yang masing-masing terhubung ke *aileron*, *elevator*, dan *rudder*. Ketika autopilot mendeteksi adanya kondisi tidak seimbang pada salah satu sumbu, sistem akan memberi perintah gerak pada servo terkait untuk mengubah posisi bidang kendali ke arah yang diperlukan hingga pesawat kembali berada dalam kondisi trim (PT Dirgantara Indonesia, n.d.).

2.3.1 Mode *Half Bank* Sistem *Autopilot* AP500R

Mode *Half Bank* pada sistem *autopilot* AP500R berbeda dari mode lateral penerbangan lainnya karena mode ini tidak menghasilkan perhitungan lateral guidance baru, tetapi hanya membatasi besar sudut bank maksimum menjadi $12,5^\circ$. Pembatasan sudut *bank* ini digunakan untuk meningkatkan kestabilan pesawat dan kenyamanan penumpang selama penerbangan. Mode *Half Bank* dapat aktif bersamaan dengan mode lateral lain. Beberapa mode lateral dapat mengaktifkan *Half Bank* secara otomatis, sementara mode lainnya justru membatalkannya. Mode ini terutama berguna pada fase jelajah sehingga *Flight Guidance Computer* (AP500R) akan memilih dan mengaktifkannya secara otomatis ketika pesawat

melewati ketinggian transisi *Half Bank* yang telah ditetapkan (PT Dirgantara Indonesia, n.d.).

2.4 Persamaan Gerak Lateral Pesawat

Dinamika lateral direksional pesawat CN235-220 dimodelkan sebagai sistem *Linear Time-Invariant* (LTI) dalam bentuk *state space*. Model ini merepresentasikan respons pesawat terhadap gangguan dan input kendali di sekitar titik operasi (*trim condition*). Persamaan gerak dinyatakan dalam bentuk persamaan diferensial matriks orde satu sebagai berikut.

$$\dot{x}(t) = A_{lat}x(t) + B_{lat}u(t) \quad (2.1)$$

$$y(t) = C_{lat}x(t) + D_{lat}u(t) \quad (2.2)$$

dengan

A_{lat} adalah matriks sistem (*system matrix*) berdimensi 4x4

B_{lat} adalah matriks input (*input matrix*) berdimensi 4x2

$x(t)$ adalah vektor keadaan (*state vector*) berdimensi 4x1

$u(t)$ adalah vektor input kendali (*control input vector*) berdimensi 2x1

2.4.1 Definisi Vektor State dan Input

Dinamika lateral direksional pesawat CN235-220 yang akan dimodelkan dalam simulasi MATLAB, vektor keadaan $x(t)$ didefinisikan oleh empat variabel utama yang merepresentasikan gerakan pesawat pada sumbu X (*roll*), dan Z (*yaw*). Empat *state* variabel tersebut dinyatakan sebagai berikut.

$$x(t) = [\rho, \beta, \dot{\beta}, r]^T \quad (2.3)$$

dengan

ρ adalah laju *roll* (*roll rate*) dalam satuan rad/s

β adalah sudut slip (*sideslip angle*) dalam satuan radian

$\dot{\beta}$ adalah laju perubahan sudut slip (*sideslip rate*) dalam satuan rad/s

r adalah laju *yaw* (*yaw rate*) dalam satuan rad/s

Vektor input kendali $u(t)$ didefinisikan sebagai defleksi *primary control surface* untuk gerak lateral pesawat. Variabel tersebut dinyatakan sebagai berikut.

$$u(t) = [\delta_a, \delta_r]^T \quad (2.4)$$

dengan

δ_a adalah defleksi aileron dalam satuan radian

δ_r adalah defleksi rudder dalam satuan radian

2.4.2 Matriks Dinamika Lateral Pesawat CN235-220

Nilai elemen matriks sistem A_{lat} dan matriks input B_{lat} diperoleh dari data teknikal PT Dirgantara Indonesia berdasarkan data hasil terbang nyata yang kemudian dilinearisasi menjadi model.

Matriks sistem A_{lat} merepresentasikan karakteristik kestabilan aerodinamika lateral pesawat. Didefinisikan sebagai berikut.

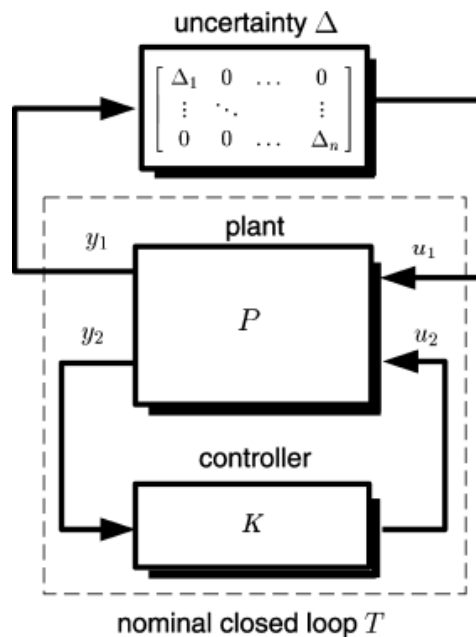
$$A_{lat} = \begin{bmatrix} -1.5 & 0.12 & 0 & -0.9 \\ 0 & -0.3 & 1 & 0.05 \\ 8 & 0 & -3 & 1 \\ 3 & 0 & 0 & -0.8 \end{bmatrix} \quad (2.5)$$

Matriks input B_{lat} merepresentasikan efektivitas aktuator. Didefinisikan sebagai berikut.

$$B_{lat} = \begin{bmatrix} 1.2 & 0.1 \\ 0 & 0.05 \\ 0.1 & 1.5 \\ 0.05 & 0.8 \end{bmatrix} \quad (2.6)$$

2.5 Kontrol Robust

Kontrol *robust* adalah pendekatan pengendalian yang digunakan pada plant atau sistem dengan dinamika yang tidak sepenuhnya terdefinisi atau yang terpengaruh oleh *disturbansi* yang tidak diketahui. Alasan utama penggunaan kontrol robust adalah keberadaan ketidakpastian (*uncertainty*) dan kebutuhan agar sistem kontrol mampu menangani ketidakpastian tersebut. *Uncertainty* dalam sistem dapat muncul dalam tiga bentuk yaitu ketidakpastian pada model plant, *disturbansi* yang memengaruhi plant, dan *noise* pada sinyal yang dibaca sensor. Setiap jenis *uncertainty* ini dapat muncul dalam bentuk aditif maupun multiplikatif. (Chandrasekharan, 1996).



Gambar 2.6 Plant Loop Kontrol dengan *Uncertainty*
(sumber: Chandrasekharan, 1996)

Diagram menampilkan interaksi antara *plant* (P) sebagai model dinamika pesawat, *controller* (K) sebagai pengendali sistem, dan blok *uncertainty* (Δ) yang mewakili gangguan eksternal. Struktur umpan balik ini, yang dikenal sebagai *Linear Fractional Transformation* menunjukkan mekanisme bagaimana *controller* harus menjaga kestabilan dan performa sistem meskipun terdapat pengaruh gangguan dari blok *uncertainty* tersebut.

Metode kontrol *robust* berusaha membatasi (*bounding*) *uncertainty*, bukan mengekspresikannya dalam bentuk distribusi probabilitas. Dengan memberikan batas pada *uncertainty*, sistem kontrol dapat memastikan bahwa kinerja yang dihasilkan tetap memenuhi persyaratan dalam seluruh kondisi yang mungkin terjadi. Oleh karena itu, kontrol *robust* lebih berorientasi pada *worst case analysis* dibandingkan pendekatan yang hanya mempertimbangkan kondisi tipikal. Tetapi, perlu dipahami bahwa beberapa aspek performa mungkin harus dikompromikan demi memastikan sistem tetap memenuhi persyaratan atau batasan kritis terutama pada sistem yang memiliki tingkat keselamatan tinggi (Chandrasekharan, 1996).

2.5.1 Linear Matrix Inequality (LMI)

Dalam perancangan sistem kontrol *robust* didapat tantangan utama menemukan parameter kontrol yang mampu memenuhi berbagai tujuan sekaligus, seperti menjamin kestabilan manuver penerbangan sekaligus membatasi energi kendali. Untuk menyelesaikan masalah multi objektif secara matematis, studi ini menggunakan pendekatan *Linear Matrix Inequality* (LMI). LMI adalah bentuk pertidaksamaan matriks yang melibatkan variabel-variabel keputusan dalam fungsi linear, yang mengubah masalah perancangan kontrol non-linear menjadi masalah optimisasi konveks (Scherer & Weiland, 2015).

Secara spesifik dalam simulasi studi yang dilakukan, LMI bekerja sebagai fondasi kestabilan. Metode yang digunakan mensintesis *gain* (K) kontroler dengan mencari matriks Lyapunov P (matriks simetris positif) yang memenuhi syarat kestabilan sistem tertutup sebagai berikut.

$$A_{cl}P + PA_{cl}^T < 0 \quad (2.7)$$

Persamaan merepresentasikan syarat kestabilan Lyapunov untuk sistem *closed-loop*. Pertidaksamaan ini menjamin kestabilan asimtotik jika terdapat matriks simetris *positive definite* P yang memenuhi kondisi tersebut. Keberadaan matriks P yang valid memastikan bahwa energi sistem akan meluruh menuju titik keseimbangan setiap kali terjadi gangguan pada pesawat. Karena hubungan antara parameter *gain* kontroler (K) dan matriks sistem aslinya bersifat non-linear, digunakan teknik substitusi variabel $Y = K.P$ untuk melinearkan masalah tersebut sehingga dapat diselesaikan secara komputasi.

Dalam implementasinya pada simulasi MATLAB, formulasi LMI diterjemahkan menggunakan *toolbox* YALMIP sebagai perumus masalah dan diselesaikan oleh *solver* numerik MOSEK. Proses ini memungkinkan sistem untuk secara otomatis menghitung kombinasi *gain* yang secara matematis stabil dan *robust* terhadap *uncertainty*.

2.5.2 Norma H_2

Norma H_2 dalam sistem kendali digunakan untuk mengukur energi respons sistem terhadap gangguan impuls atau *white noise*. Tujuan utama dari sintesis H_2

adalah meminimalkan energi rata-rata dari variabel keluaran. Dalam studi ini meminimalkan norma H_2 setara dengan.

- Meminimalkan energi side slip
- Meminimalkan energi percepatan lateral
- Meminimalkan energi sinyal kendali (gerak aktuator)

Secara matematis masalah ini diselesaikan menggunakan Linear Matrix Inequality (LMI). Sistem dinyatakan memiliki performa H_2 yang lebih kecil dari nilai v jika terdapat matriks simetris positif P dan matriks W yang memenuhi persamaan kestabilan sistem berikut.

$$A_{cl}P + PA_{cl}^T + B_u Y + Y^T B_u^T < 0 \quad (2.8)$$

dengan batas energi H_2 *performance* memiliki syarat yang perlu dipenuhi sebagai berikut.

$$Trace < v^2 \quad (2.9)$$

W adalah matriks bantu untuk membatasi energi keluaran. Dengan meminimalkan $Trace(W)$ didapatkan nilai *gain* (K) yang menghasilkan respons sistem paling sesuai dengan tujuan optimisasi *gain* (Shinners, 1998).

2.5.3 Norma H_∞

Norma H_∞ (*H-infinity*) dalam sistem kendali digunakan untuk mengukur maksimum *gain* dari gangguan terhadap keluaran sistem di seluruh frekuensi merepresentasikan respons sistem terhadap skenario *worst case*. Norma H_∞ berfungsi sebagai jaminan keselamatan. Tujuannya memastikan bahwa meskipun pesawat mengalami *uncertainty* ekstrem, sistem tidak akan menjadi tidak stabil atau mengalami deviasi yang berbahaya.

Masalah ini diformulasikan untuk menjaga norma H_∞ di bawah batas nilai γ . Berdasarkan *Bounded Real Lemma*, hal ini terpenuhi jika terdapat matriks P yang memenuhi LMI berikut.

$$\begin{bmatrix} AP + PA^T + B_u Y + Y^T B_u^T & * & * \\ B_\infty^T & -\gamma I & * \\ C_\infty P + D_{\infty u} Y & D_{\infty \infty} & -\gamma I \end{bmatrix} < 0 \quad (2.10)$$

Nilai γ yang kecil menunjukkan sistem memiliki *robustness* yang tinggi terhadap *uncertainty* (Shinners, 1998).

2.6 Analisis Pengendalian Sistem

Mengendalikan sebuah sistem terdapat tiga hal utama yang menjadi perhatian *observability*, *controllability*, dan *stability*. *Observability* adalah kemampuan untuk mengamati semua parameter atau variabel keadaan (*state variables*) dalam sistem. *Controllability* adalah kemampuan untuk memindahkan sistem dari kondisi awal tertentu menuju kondisi yang diinginkan. *Stability* sering didefinisikan sebagai respons sistem yang tetap terbatas (*bounded*) terhadap setiap masukan yang juga terbatas (*bounded input*). Sistem kontrol yang sukses harus memiliki dan mempertahankan ketiga sifat ini. *Uncertainty* menjadi tantangan bagi sistem kontrol dalam mempertahankan sifat-sifat tersebut (Lewis, 1986).

Sistem *controllable* ketika keluaran yang diinginkan di peroleh dengan menerapkan masukan terkendali. Merupakan kemampuan untuk mengendalikan keadaan sistem. Dengan persamaan sebagai berikut.

$$Q_0 = [B \ AB \ A^2B \ \dots \ A^{(n-1)}B] \quad (2.11)$$

dengan Q_0 memiliki syarat yang perlu dipenuhi sebagai berikut.

$$|Q_0| \neq 0 \text{ (Sistem controllable)}$$

$$|Q_0| = 0 \text{ (Sistem uncontrollable)}$$

Sistem *observable* ketika mengukur atau mengamati keadaan sistem dapat dilakukan. Jika keadaan internal sistem dapat ditentukan menggunakan sinyal masukan dan keluaran selama interval waktu yang terbatas, maka sistem tersebut dikatakan dapat diamati. Dengan persamaan sebagai berikut.

$$Q_0 = [C^T \ A^T C^T \ \dots \ (A^T)^{n-1} C^T] \quad (2.12)$$

dengan Q_0 memiliki syarat yang perlu dipenuhi sebagai berikut.

$$|Q_0| \neq 0 \text{ (Sistem observable)}$$

$$|Q_0| = 0 \text{ (Sistem unobservable)}$$

2.7 Optimisasi Quasi Newton

Dalam perancangan sistem kendali terbang diperlukan pendekatan yang menjamin kestabilan matematis sekaligus pencapaian performa operasional manuver penerbangan yang dibutuhkan. Untuk memenuhi tujuan tersebut studi menerapkan metode optimisasi Quasi Newton yang dimodifikasi agar dapat bekerja secara hibrida dengan *constraint* norma H_2 dan norma H_∞ yang diformulasikan.

2.7.1 Algoritma Quasi Newton BFGS

Algoritma Quasi Newton varian BFGS (Broyden–Fletcher–Goldfarb–Shanno) digunakan untuk mencari nilai optimal *gain* (K) kontroler terbang. Metode ini dipilih karena efisiensinya dalam menyelesaikan masalah optimisasi non-linear *unconstrained optimization* dengan cara mencari titik minimum dari sebuah fungsi biaya (*J*) (Shinners, 1998). Mekanisme pembaruan *gain* kontroler (K) pada setiap iterasi (*i*) diformulasikan sebagai berikut.

$$K_{i+1} = K_i - \alpha \cdot H_i^{-1} \cdot \nabla J(K_i) \quad (2.13)$$

dengan

K adalah nilai parameter *gain* kontroler yang sedang dioptimasi

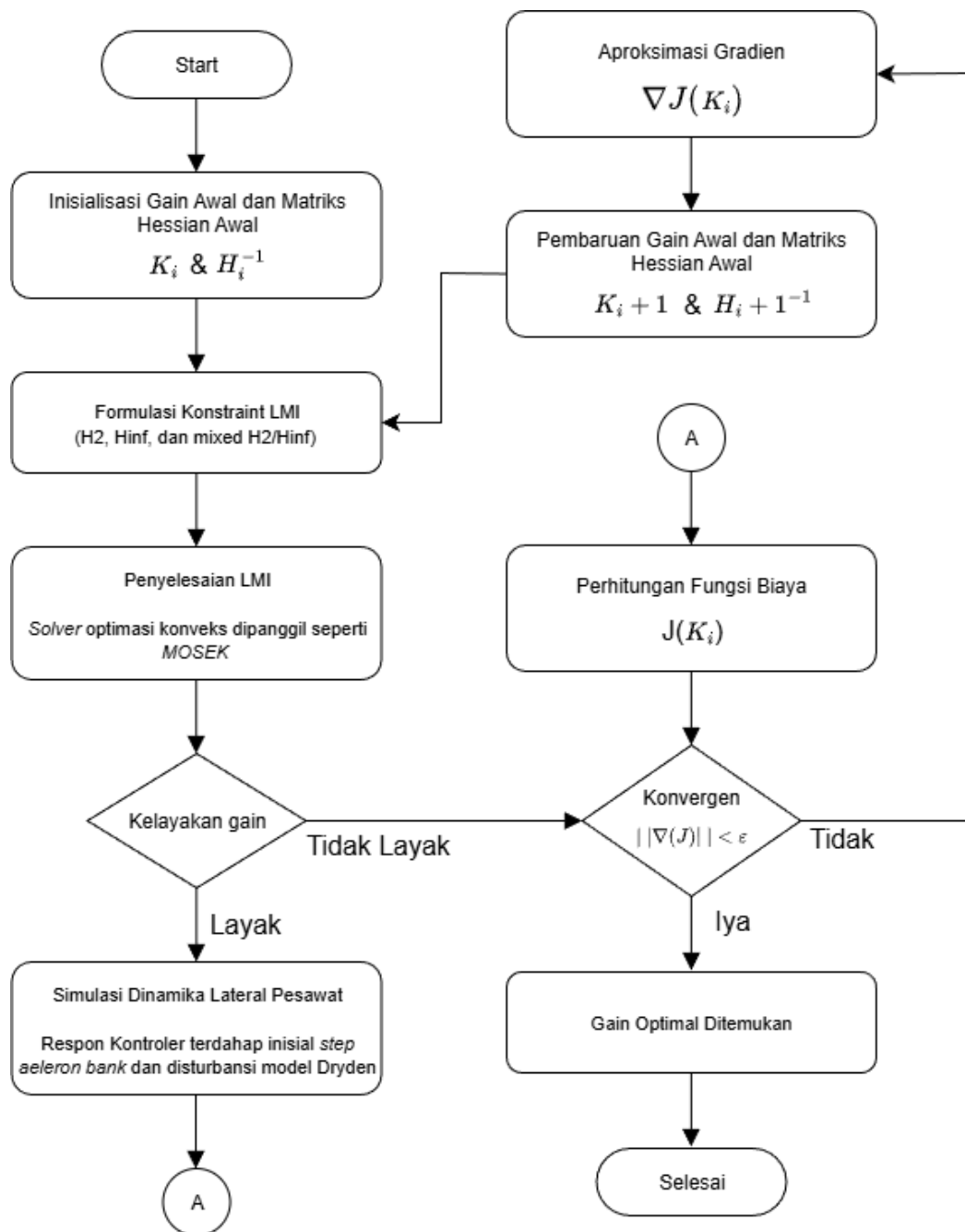
α adalah ukuran langkah (*step size*) yang menentukan besar perubahan nilai *gain* pada setiap iterasi

$\nabla J(K_i)$ adalah gradien fungsi biaya yang menunjukkan arah kenaikan/penurunan nilai *error*

H_i^{-1} adalah matriks aproksimasi invers Hessian yang diperbarui secara iteratif untuk memperbaiki arah pencarian agar hasil tetap konvergen

2.7.2 Batasan Kestabilan Kontrol *Robust*

Meskipun algoritma Quasi Newton BFGS digunakan untuk mencari nilai optimal *gain* (K), algoritma ini tidak secara inheren menjamin kestabilan sistem kendali dan memenuhi pengendalian secara *robust*. Dalam struktur optimisasi hibrida yang dirancang, setiap kali algoritma Quasi Newton BFGS mengusulkan nilai *gain* baru (K_i) akan dilakukan pemeriksaan kelayakan (*feasibility check*). Secara alur digambarkan pada Gambar 2.7 sebagai berikut.



Gambar 2.7 Diagram Alir Optimasi Dinamika Lateral Pesawat

Pemeriksaan menggunakan *Linear Matrix Inequality* (LMI) sebagai filter. Jika sebuah set *gain* menghasilkan sistem yang tidak stabil (LMI *infeasible*), maka *gain* tersebut ditolak dan optimisasi dipaksa untuk mengurangi ukuran langkah (*step size*) atau mencari arah lain, tanpa perlu menjalankan simulasi. Ini memastikan bahwa nilai *gain* (K) optimal dapat berjalan didalam *constraint* kontrol secara *robust*.

BAB III

HASIL PEMBELAJARAN

3.1 Deskripsi dan Tujuan Tugas Khusus

Tugas khusus kerja praktik berfokus pada perancangan dan pengembangan sistem kendali *state feedback* untuk mengoptimalkan manuver belok terkoordinasi (*coordinated turn*) mode *half bank* pada pesawat CN235-220, menggunakan data performa nyata pesawat. Sistem kendali dirancang untuk menjaga stabilitas dan performa dinamika lateral pesawat dengan permodelan dan simulasi dilakukan di software MATLAB.

Metode yang digunakan menggabungkan metode control robust berbasis *Linear Matrix Inequality* (LMI) pada *inner loop* dengan algoritma *quasi-Newton* pada *outer loop*. Pendekatan ini digunakan untuk mensintesis dan membandingkan empat strategi kendali.

- a. Kendali awal, Model kontrol pesawat dasar untuk pesawat CN235-220 tanpa gain optimization.
- b. Kendali H_2 murni, berfokus pada performa optimal dalam meredam gangguan eksternal dan meminimalkan energi kendali.
- c. Kendali H_∞ murni, menekankan ketahanan (*robustness*) terhadap ketidakpastian dan gangguan eksternal.
- d. Kendali campuran H_2/H_∞ , pendekatan multi-objektif yang menyeimbangkan tuntutan performa H_2 dan ketahanan H_∞ dalam satu pengendali.

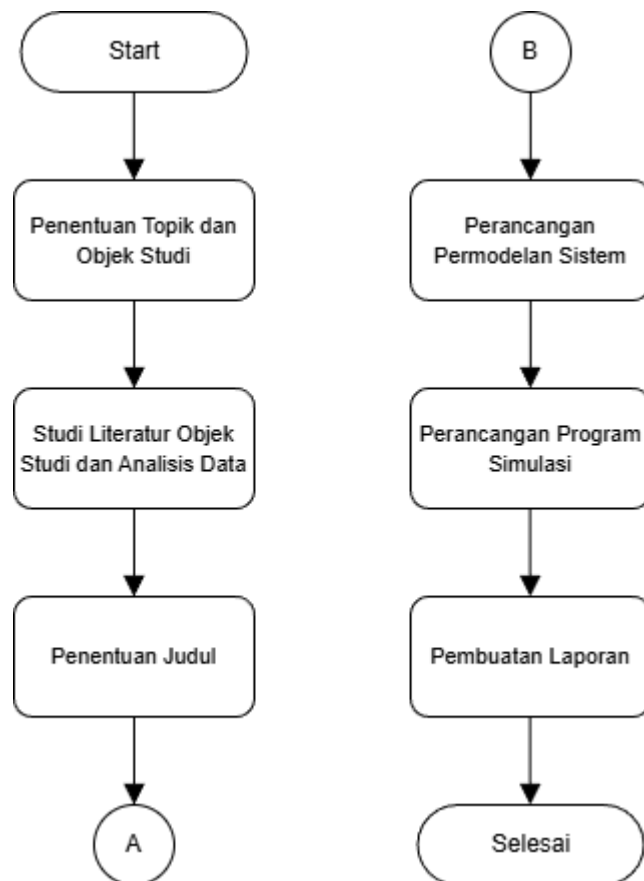
gangguan eksternal atau *uncertainty* dimodelkan menggunakan model matematis turbulensi angin Dryden. Optimisasi difokuskan pada mencari *gain* kendali yang meminimalkan nilai error *slip* dan *skid* pesawat dengan respons empat variabel utama dinamika lateral ρ (laju *roll*), β (sudut *sideslip*), $\dot{\beta}$ (laju *sideslip*), dan r (laju *yaw*) saat menerima perintah belokan *aileron*.

Tujuan akhir tugas khusus kerja praktik adalah menghasilkan rancangan sistem kendali untuk meminimalkan deviasi pergerakan pada sudut *sideslip* (β) sehingga pergerakan lateral pesawat tetap terkoordinasi tanpa adanya *slip* dan *skid* selama manuver belokan, dilakukan juga analisis luaran pendukung seperti

percepatan lateral (a_l) dan waktu defleksi bidang kendali pesawat untuk memperkuat konklusi kerja praktik yang dilakukan. Hasil kerja praktik diharapkan dapat menjadi acuan bagi PT Dirgantara Indonesia untuk meningkatkan performa manuver lateral pada sistem *autopilot* CN235-220.

3.2 Metode Penyelesaian Tugas Khusus

Metode penyelesaian tugas khusus kerja praktik disusun secara sistematis untuk mendukung perancangan dan pengembangan sistem kendali *state feedback* dalam rangka mengoptimalkan manuver belok terkoordinasi (*coordinated turn*) pada mode *half bank* pesawat CN235-220. Secara alur metode penyelesaian tugas khusus digambarkan pada Gambar 3.1 sebagai berikut.



Gambar 3.1 Diagram Alir Metode Penyelesaian Tugas Khusus

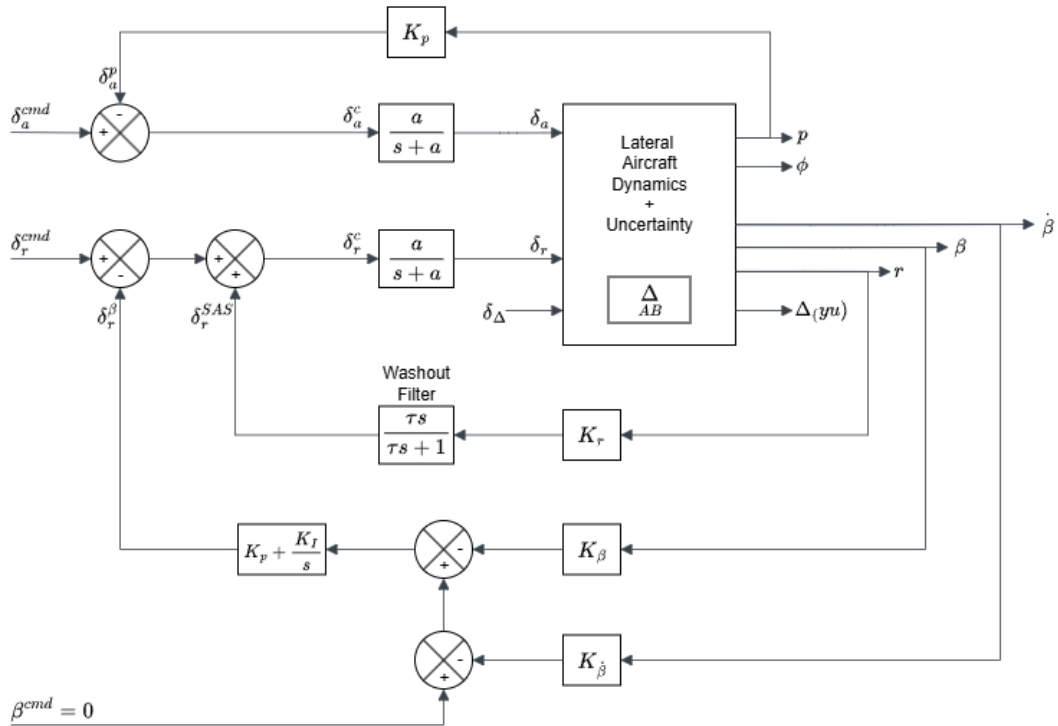
Tahapan penyelesaian dirancang secara terintegrasi, diawali dengan penetapan topik kerja praktik pada bidang sistem *autopilot* dengan fokus pada pemodelan dinamika lateral. Pemilihan objek studi didasarkan pada ketersediaan sistem *autopilot* dan kelengkapan data uji yang diperlukan untuk memperoleh model yang

representatif. Berdasarkan pertimbangan tersebut, pesawat CN235-220 ditetapkan sebagai objek kajian mengingat statusnya sebagai salah satu produk awal rancangan PT Dirgantara Indonesia yang telah terbukti keandalannya. Latar belakang pemilihan topik ini adalah kompleksitas pemodelan dinamika lateral yang memerlukan formulasi model matematis akurat guna merepresentasikan kinerja dinamik pesawat secara realistis. Model yang dihasilkan selanjutnya digunakan dalam proses optimisasi untuk peningkatan performa dinamika lateral. Setelah topik dan objek studi ditetapkan, dilakukan studi literatur dari sumber tepercaya, termasuk dokumen teknis internal PT Dirgantara Indonesia. Selanjutnya, data uji nyata pesawat CN235-220 dianalisis dan diolah sebagai dasar dalam proses pemodelan dinamika lateral sistem.

Tahap berikutnya adalah penentuan judul yang telah disesuaikan dengan topik terpilih yaitu optimisasi dinamika lateral *response* pada pesawat. Dirumuskan pendekatan kendali *robust* optimisasi campuran H_2/H_∞ yang memanfaatkan formulasi *Linear Matrix Inequality* (LMI) dan algoritma *quasi-Newton* yang dirancang merespon secara komplementer, membentuk skema optimisasi bertingkat yang terdiri atas *inner loop* dan *outer loop* dengan tujuan kinerja optimisasi yang saling mendukung. Dengan formulasi judul beserta fokus topik yang telah ditetapkan menjadi dasar tahap perancangan permodelan sistem.

3.3 Permodelan Belok Terkoordinasi

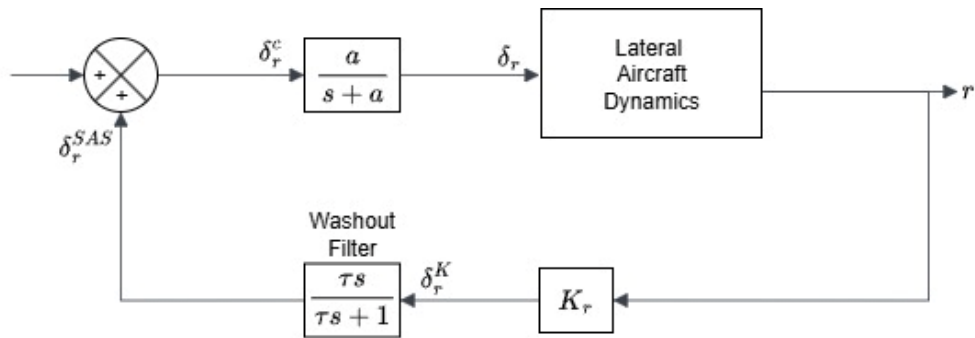
Kontrol belok terkoordinasi (*turn coordination*) dirancang untuk menjaga keseimbangan aerodinamis pesawat selama manuver *half bank turn* dengan memastikan interaksi yang tepat antara aktuator *rudder* dan *aileron*. Tujuan utamanya adalah meminimalkan deviasi sudut *sideslip* (β) sehingga pesawat dapat melakukan belokan secara terkoordinasi tanpa terjadinya *slip* dan *skid*. Arsitektur sistem kendali terbang telah dirancang terlihat pada Gambar 3.1, yang menjadi dasar perancangan *state feedback controller* dalam kerja praktik ini. Terdiri dari *rudder channel* yang terbentuk dari gabungan *inner loop* dengan respons cepat untuk meredam laju *yaw* (*damping controller*), dan *outer loop* dengan respons lambat untuk kontrol *sideslip*. Selain itu, *aileron channel loop* untuk kontrol *roll*.



Gambar 3.2 Blok Diagram Sistem Belok Terkoordinasi Keseluruhan

3.3.1 Kontroler Yaw

Sistem kendali yaw menerapkan struktur *Stability Augmentation System* (SAS) sebagai *inner loop*. Struktur diagram blok sistem tersebut digambarkan pada Gambar 3.3 sebagai berikut.



Gambar 3.3 Blok Diagram Loop Dalam (*Inner Loop*) Kontrol Yaw

Formulasi dari respon kontroler laju yaw atau dalam konteks keseluruhan sistem belok terkoordinasi disebut kontrol loop dalam (*inner loop*) *damping* kontroler adalah sebagai berikut.

$$\frac{\delta_r^{SAS}}{\delta_r^K} = \frac{\tau s}{\tau s + 1} = \tau s \cdot \frac{1}{\tau s + 1} = \frac{\delta_r^{SAS}}{x_{wof}} \cdot \frac{x_{wof}}{K_r r(t)}$$

Jika diambil bentuk $\frac{x_{wof}}{K_r r(t)}$

$$\frac{x_{wof}}{K_r r(t)} = \frac{1}{\tau s + 1} \rightarrow \tau s x_{wof} + x_{wof} = K_r r(t)$$

Inverse Laplace transform menghasilkan.

$$\dot{x}_{wof}(t) = \frac{-x_{wof}}{\tau} + \frac{K_r r(t)}{\tau} \quad (3.1)$$

Jika diambil bentuk $\frac{\delta_r^{SAS}}{x_{wof}}$

$$\frac{\delta_r^{SAS}}{x_{wof}} = \tau s \rightarrow \tau \dot{x}_{wof}(t) = \delta_r^{SAS}$$

Disubstitusikan $x_{wof}(t)$ didapatkan.

$$\tau \left(\frac{-x_{wof}(t)}{\tau} + \frac{K_r r(t)}{\tau} \right) = \delta_r^{SAS}$$

Atau

$$\delta_r^{SAS} = -x_{wof}(t) + K_r r(t) \quad (3.2)$$

Persamaan (3.1) dan (3.2) kemudian diubah menjadi bentuk *state space* menghasilkan persamaan.

$$\dot{x}_{wof}(t) = \left[\frac{-1}{\tau} \right] x_{wof}(t) + \left[\frac{K_r}{\tau} \right] r(t) \quad (3.3)$$

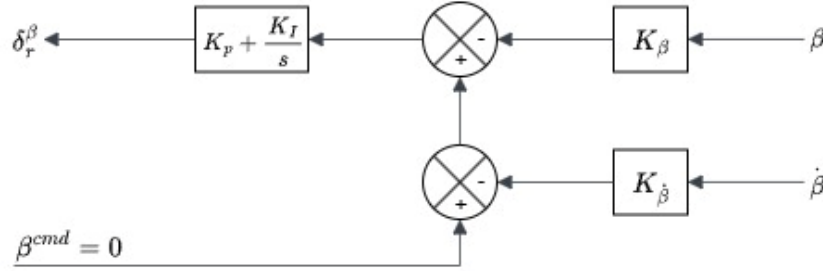
$$\delta_r^{SAS} = [-1]x_{wof}(t) + [K_r]r(t) \quad (3.4)$$

dengan

- x_{wof} merepresentasikan keadaan internal *filter* atau *damper yaw* dengan waktu-respon τ . Model ini adalah *filter first-order* yang menyaring sinyal *yaw rate* $r(t)$ sebelum menghasilkan aksi kendali.
- Koefisien K_r menentukan seberapa agresif *SAS* merespon perubahan *yaw rate*. Bentuk keluaran $\delta_r^{SAS} = -x_{wof}(t) + K_r r(t)$ menunjukkan kombinasi *feedforward* dan *feedback filter* untuk menambah redaman pada *yaw*.
- Tujuan mengimplementasikan *inner loop* ini meredam osilasi *yaw* (*Dutch roll/spiral*) dan menjaga *yaw rate* stabil. Karena sifatnya kritis untuk stabilitas lateral, *loop* ini dibuat lebih cepat (*bandwidth* lebih tinggi) dari *loop sideslip*.

3.3.2 Kontroler *Sideslip*

Sistem kendali sideslip beroperasi sebagai outer loop untuk meminimalkan deviasi sudut sideslip selama manuver pesawat. Struktur diagram blok sistem tersebut digambarkan pada Gambar 3.4 sebagai berikut.



Gambar 3.4 Blok Diagram Loop Luar (*Outer Loop*) Kontrol *Sideslip*

Formulasi dari respon kontroler *sideslip* dan laju *sideslip* atau dalam konteks sistem belok terkoordinasi disebut kontrol *loop* luar (*outer loop*) *sideslip* kontroler adalah sebagai berikut.

$$\dot{e}_I(t) = [0]e_I(t) + [-K_\beta \quad -K_{\dot{\beta}}] \begin{bmatrix} \beta(t) \\ \dot{\beta}(t) \end{bmatrix} + [1]\beta^{cmd}(t) \quad (3.5)$$

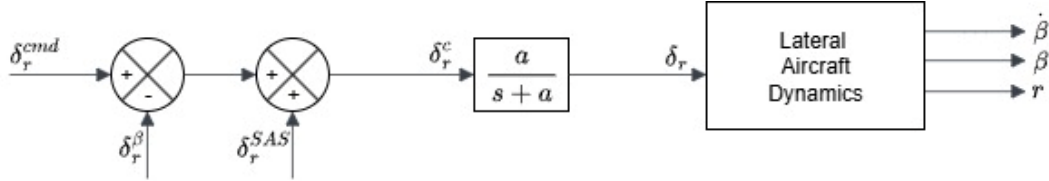
$$\delta_r^\beta = [K_I]e_I(t) + [-K_p K_\beta \quad -K_p K_{\dot{\beta}}] \begin{bmatrix} \beta(t) \\ \dot{\beta}(t) \end{bmatrix} + [K_p]\beta^{cmd}(t) \quad (3.6)$$

dengan

- e_I merepresentasikan *integrator error* yang mengumpulkan deviasi *sideslip* (β) terhadap perintah $\beta^{cmd} = 0$ untuk koordinasi sempurna (tidak ada *sideslip*). *Integrator* ini memberikan aksi *steady state* untuk menghilangkan *offset sideslip* akibat *disturbance*.
- Koefisien K_β dan $K_{\dot{\beta}}$ menentukan gain umpan balik pada posisi derajat *sideslip* dan laju perubahan *sideslip*. Sementara itu, Koefisien K_I menentukan gain *integrator* pada aksi rudder yang dihasilkan dan K_p menentukan gain proporsional pada aksi rudder yang dihasilkan.
- Tujuan mengimplementasikan *outer loop* ini berfokus pada kualitas belok (*minimalkan slip/skid*) dan kestabilan lateral jangka panjang.

3.3.3 Kontroler Rudder

Sistem kendali rudder mengintegrasikan aksi *yaw damper* dari *inner loop* dan kendali *sideslip* dari *outer loop* untuk menghasilkan sinyal defleksi total. Integrasi kedua sistem tersebut digambarkan pada Gambar 3.5 sebagai berikut.



Gambar 3.5 Blok Diagram Kontroler Rudder

Formulasi kontroler merepresentasikan satu *kontroler rudder* adalah sebagai berikut.

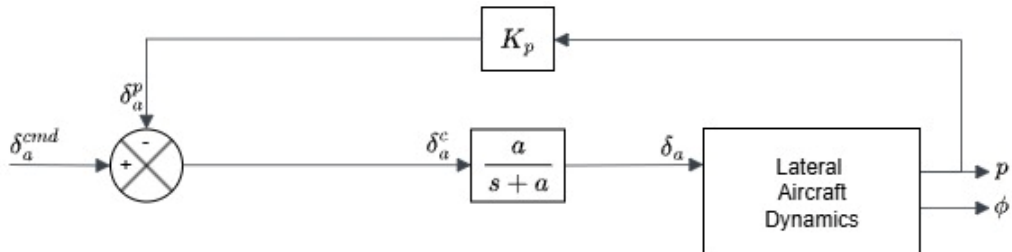
$$\delta_r^c = \delta_r^{cmd} + \delta_r^\beta + \delta_r^{SAS} \quad (3.7)$$

Persamaan tersebut adalah total perintah rudder δ_r^c superposisi dari tiga kontribusi sebagai berikut.

- δ_r^{cmd} merupakan perintah utama dari pengendali *yaw* atau *autopilot yaw* channel,
- δ_r^β merupakan aksi untuk mengatur *sideslip* (outer loop),
- δ_r^{SAS} merupakan aksi *damping* dari SAS (inner loop).

3.3.4 Kontroler Aileron

Sistem kendali aileron berfungsi mengatur dinamika laju *roll* pesawat melalui mekanisme umpan balik proporsional. Struktur diagram blok sistem tersebut digambarkan pada Gambar 3.6 sebagai berikut.



Gambar 3.6 Blok Diagram Kontroler Aileron

Formulasi dari respon kontroler laju *roll* merepresentasikan satu kontroler aileron adalah sebagai berikut.

$$\delta_a^c = \delta_a^{cmd} - \delta_a^p \quad (3.8)$$

$$\delta_a^c = \delta_a^{cmd} - K_\rho \rho(t) \quad (3.9)$$

Persamaan tersebut adalah total perintah aileron δ_a^c superposisi dari dua kontribusi sebagai berikut.

- δ_a^{cmd} merupakan perintah aileron utama dari pengendali *roll* atau *autopilot roll* channel,
- δ_a^p merupakan aksi untuk mengatur dinamika *roll*.

3.3.5 Representasi Model Belok Terkoordinasi

Representasi *state space* keseluruhan untuk model belok terkoordinasi pesawat dapat diformulasikan menggunakan Persamaan (2.1) dan Persamaan (2.2) dengan memasukan matriks yang telah ditetapkan pada Persamaan (2.5) dan Persamaan (2.6). Dituliskan sebagai berikut.

$$\begin{bmatrix} \dot{\rho}(t) \\ \dot{\beta}(t) \\ \ddot{\beta}(t) \\ \dot{r}(t) \end{bmatrix} = \begin{bmatrix} -1.5 & 0.12 & 0 & -0.9 \\ 0 & -0.3 & 1 & 0.05 \\ 8 & 0 & -3 & 1 \\ 3 & 0 & 0 & -0.8 \end{bmatrix} \begin{bmatrix} \rho \\ \beta \\ \dot{\beta} \\ r \end{bmatrix} + \begin{bmatrix} 1.2 & 0.1 \\ 0 & 0.05 \\ 0.1 & 1.5 \\ 0.05 & 0.8 \end{bmatrix} \begin{bmatrix} \delta_a \\ \delta_r \end{bmatrix} \quad (3.10)$$

$$\begin{bmatrix} \rho(t) \\ \beta(t) \\ \dot{\beta}(t) \\ r(t) \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \rho \\ \beta \\ \dot{\beta} \\ r \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} \delta_a \\ \delta_r \end{bmatrix} \quad (3.11)$$

Dengan mengabungkan formulasi dari persamaan (3.7) kontroler rudder, dan persamaan (3.9) kontroler aileron menjadi bentuk *state space* didapatkan persamaan model sebagai berikut.

Model Keadaan Sistem

$$\begin{bmatrix} \dot{e}_I(t) \\ \dot{x}_{wof}(t) \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & -\frac{1}{\tau} \end{bmatrix} \begin{bmatrix} e_I(t) \\ x_{wof}(t) \end{bmatrix} + \begin{bmatrix} 0 & -K_\beta & -K_{\dot{\beta}} & 0 \\ 0 & 0 & 0 & \frac{Kr}{\tau} \end{bmatrix} \begin{bmatrix} \rho(t) \\ \beta(t) \\ \dot{\beta}(t) \\ r(t) \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \beta(t) \\ \delta_a^{cmd}(t) \\ \delta_r^{cmd}(t) \end{bmatrix} \quad (3.12)$$

Model Kontroler Sistem

$$\begin{bmatrix} \delta_a^c \\ \delta_r^c \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ K_I & -1 \end{bmatrix} \begin{bmatrix} e_I(t) \\ x_{wof}(t) \end{bmatrix} + \begin{bmatrix} -K_\rho & 0 & 0 & 0 \\ 0 & -K_\rho K_\beta & -K_\rho K_{\dot{\beta}} & K_r \end{bmatrix} \begin{bmatrix} \rho(t) \\ \beta(t) \\ \dot{\beta}(t) \\ r(t) \end{bmatrix} + \begin{bmatrix} 0 & 1 & 0 \\ K_\rho & 0 & 1 \end{bmatrix} \begin{bmatrix} \beta(t) \\ \delta_a^{cmd}(t) \\ \delta_r^{cmd}(t) \end{bmatrix} \quad (3.13)$$

3.4 Analisis Controllability dan Observability Permodelan

Analisis *controllability* dilakukan menggunakan Persamaan (2.11) dengan representasi *state space* pesawat menggunakan Persamaan (3.10). Karena matrix A memiliki dimensi 4×4 atau $n = 4$, maka akan menganalisis nilai matrix hingga A^3B .

$$Q_0 = [B \mid AB \mid A^2B \mid A^3B]$$

Menghitung kolom pertama – B

$$B = \begin{bmatrix} 1.2 & 0.1 \\ 0 & 0.05 \\ 0.1 & 1.5 \\ 0.05 & 0.8 \end{bmatrix}$$

Menghitung kolom kedua – AB

$$AB = \begin{bmatrix} -1.5 & 0.12 & 0 & -0.9 \\ 0 & -0.3 & 1 & 0.05 \\ 8 & 0 & -3 & 1 \\ 3 & 0 & 0 & -0.8 \end{bmatrix} \begin{bmatrix} 1.2 & 0.1 \\ 0 & 0.05 \\ 0.1 & 1.5 \\ 0.05 & 0.8 \end{bmatrix}$$

$$AB = \begin{bmatrix} -1.845 & -0.864 \\ 0.1025 & 1.525 \\ 9.35 & -2.9 \\ 3.56 & -0.34 \end{bmatrix}$$

Menghitung kolom ketiga – $A^2B = A \times AB$

$$A^2B = \begin{bmatrix} -1.5 & 0.12 & 0 & -0.9 \\ 0 & -0.3 & 1 & 0.05 \\ 8 & 0 & -3 & 1 \\ 3 & 0 & 0 & -0.8 \end{bmatrix} \begin{bmatrix} -1.845 & -0.864 \\ 0.1025 & 1.525 \\ 9.35 & -2.9 \\ 3.56 & -0.34 \end{bmatrix}$$

$$A^2B = \begin{bmatrix} -0.4542 & 1.785 \\ 9.49725 & -3.3745 \\ -39.09 & 1.448 \\ -8.323 & -2.32 \end{bmatrix}$$

Menghitung kolom keempat – $A^3B = A \times A^2B$

$$A^3B = \begin{bmatrix} -1.5 & 0.12 & 0 & -0.9 \\ 0 & -0.3 & 1 & 0.05 \\ 8 & 0 & -3 & 1 \\ 3 & 0 & 0 & -0.8 \end{bmatrix} \begin{bmatrix} -0.4542 & 1.785 \\ 9.49725 & -3.3745 \\ -39.09 & 1.448 \\ -8.323 & -2.32 \end{bmatrix}$$

$$A^3B = \begin{bmatrix} 9.31167 & -0.99444 \\ -42.355325 & 2.34435 \\ 105.3134 & 7.616 \\ 5.2958 & 7.211 \end{bmatrix}$$

Menghitung determinan - $|Q_0|$

$$|Q_0| = [B \mid AB \mid A^2B \mid A^3B]$$

$$|Q_0|$$

$$= \begin{bmatrix} 1.2 & 0.1 & -1.845 & -0.864 & -0.4542 & 1.785 & 9.31167 & -0.99444 \\ 0 & 0.05 & 0.1025 & 1.525 & 9.49725 & -3.3745 & -42.355325 & 2.34435 \\ 0.1 & 1.5 & 9.35 & -2.9 & -39.09 & 1.448 & 105.3134 & 7.616 \\ 0.05 & 0.8 & 3.56 & -0.34 & -8.323 & -2.32 & 5.2958 & 7.211 \end{bmatrix}$$

Dengan bantuan komputasi Matlab didapat sistem berkedudukan pada rank (Q_0) = 4, sistem *fully controllable*.

Analisis *observability* dilakukan menggunakan Persamaan (2.12) dengan persamaan representasi *state space* pesawat menggunakan Persamaan (3.10) dan (3.11). Karena matrix A memiliki dimensi 4×4 atau $n = 4$, maka akan menganalisis nilai matrix hingga CA^3 .

$$Q_0 = \begin{bmatrix} C \\ CA \\ CA^2 \\ CA^3 \end{bmatrix}$$

Menghitung baris pertama – C

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Menghitung baris kedua – CA

$$CA = C \begin{bmatrix} -1.5 & 0.12 & 0 & -0.9 \\ 0 & -0.3 & 1 & 0.05 \\ 8 & 0 & -3 & 1 \\ 3 & 0 & 0 & -0.8 \end{bmatrix}$$

Menghitung baris ketiga – $CA^2 = CA \times A$

$$CA^2 = \begin{bmatrix} -1.5 & 0.12 & 0 & -0.9 \\ 0 & -0.3 & 1 & 0.05 \\ 8 & 0 & -3 & 1 \\ 3 & 0 & 0 & -0.8 \end{bmatrix} \begin{bmatrix} -1.5 & 0.12 & 0 & -0.9 \\ 0 & -0.3 & 1 & 0.05 \\ 8 & 0 & -3 & 1 \\ 3 & 0 & 0 & -0.8 \end{bmatrix}$$

$$CA^2 = \begin{bmatrix} -0.45 & -0.216 & 0.12 & 2.076 \\ 8.15 & 0.09 & -3.3 & 0.945 \\ -33 & 0.96 & 9 & -11 \\ -6.9 & 0.36 & 0 & -2.06 \end{bmatrix}$$

Menghitung baris keempat – $CA^3 = CA^2 \times A$

$$CA^3 = \begin{bmatrix} -0.45 & -0.216 & 0.12 & 2.076 \\ 8.15 & 0.09 & -3.3 & 0.945 \\ -33 & 0.96 & 9 & -11 \\ -6.9 & 0.36 & 0 & -2.06 \end{bmatrix} \begin{bmatrix} -1.5 & 0.12 & 0 & -0.9 \\ 0 & -0.3 & 1 & 0.05 \\ 8 & 0 & -3 & 1 \\ 3 & 0 & 0 & -0.8 \end{bmatrix}$$

$$CA^3 = \begin{bmatrix} 7.863 & 0.0108 & -0.576 & -1.1466 \\ -35.79 & 0.951 & 9.99 & -11.3865 \\ 88.5 & -4.248 & -26.04 & 47.548 \\ 4.17 & -0.936 & 0.36 & 7.876 \end{bmatrix}$$

Dengan bantuan komputasi Matlab didapat sistem berkedudukan pada rank (Q_0) = 4, sistem *fully observable*.

3.5 Analisis Pole Placement Permodelan

Analisis *pole placement* dilakukan yang merupakan sebuah metode dalam teori kontrol modern yang menggunakan umpan balik *state variable* untuk menempatkan *poles* (kutub) dari sistem *closed loop* pada lokasi yang diinginkan di bidang-s. Dengan memindahkan *poles* merupakan *eigenvalues* dari matriks sistem dapat secara langsung mengontrol karakteristik stabilitas dan respons transien dari sistem, seperti rasio redaman (damping ratio) dan frekuensi natural.

Analisis ini dapat dilakukan karena semua variabel *state* (x) *fully controllable* dan *fully observable* ke input sistem (u) melalui sebuah matriks penguatan (gain matrix) K . Dapat dituliskan sebagai persamaan berikut.

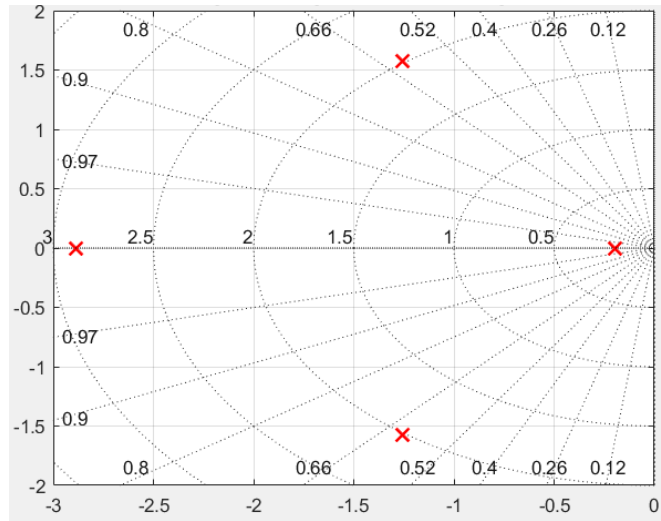
$$u = -Kx \quad (3.14)$$

Ketika hukum kontrol ini diterapkan pada sistem *open loop* $\dot{x}(t) = Ax(t) + Bu$, sistem *closed loop* yang dihasilkan menjadi.

$$\dot{x} = -Ax + B(-Kx) = (A - BK)x \quad (3.15)$$

Tujuan akhir dari analisis ini untuk menghitung matriks K yang sesuai sehingga *eigenvalues* dari matriks $(A - BK)x$ berada tepat di lokasi *poles* baru yang di tentukan dan menjadi target *gain* optimal $K\rho$ (laju *roll*), $K\beta$ (sudut *sideslip*), $K\dot{\beta}$ (laju *sideslip*), dan Kr (laju *yaw*).

3.5.1 Analisis *Open Loop* Permodelan



Gambar 3.7 *Pole Placement Open Loop* Permodelan

Analisis *inheren* dilakukan dari model plant (sistem open loop) dengan mencari *eigenvalues* dari matriks A . Menggunakan simulasi MATLAB, diperoleh empat *poles open loop* sebagai berikut.

$$p = [-2.8866, -1.2573 \pm 1.5738i, -0.1988]$$

Semua *poles* memiliki bagian riil yang negatif menunjukkan sistem *open loop* stabil. Jika dianalisis mode *lateral* direksional utama dari *poles* ini didapat.

1. Mode Dutch Roll, direpresentasikan oleh pasangan *poles* kompleks $p = -1.2573 \pm 1.5738i$. Dari *poles* ini dapat dihitung nilai sebagai berikut.

$$\text{Frekuensi Natural } (\omega_n) = \sqrt{(1.2573)^2 + (1.5738)^2} = 2.014 \text{ rad/s}$$

$$\text{Damping Ratio } (\xi) = 1.2573/2.014 = 0.624$$

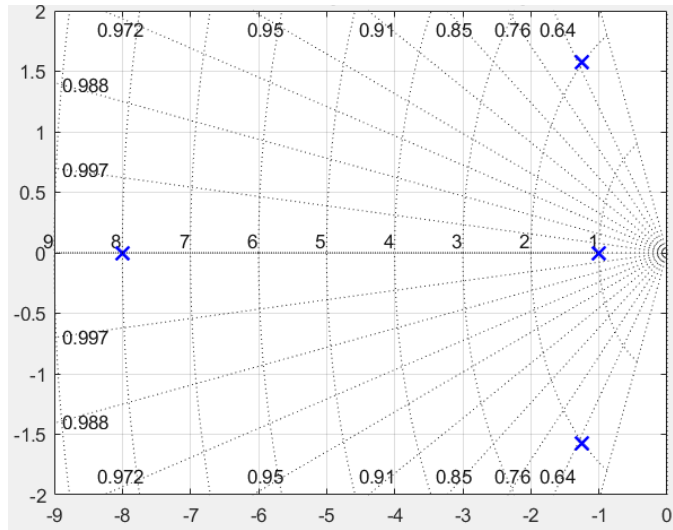
Nilai *Damping Ratio* (ξ) yang didapat memenuhi *constraint* $\xi \geq 0.1$ ini menunjukkan osilasi dutch roll alami dari pesawat sudah memiliki redaman yang sangat baik.

2. Mode Roll, direpresentasikan oleh *pole* sumbu riil -2.8866 . Dari *pole* ini dapat dihitung nilai sebagai berikut.

$$\text{Time Constant } (T_r) = -1/(-2.8866) = 0.346 \text{ s}$$

3. Mode Spiral, direpresentasikan oleh *pole* riil -0.1988 . *Pole* ini stabil (negatif), sehingga pesawat tidak memiliki tendensi divergensi spiral.

3.5.2 Analisis *Closed Loop* Permodelan



Gambar 3.8 *Pole Placement Closed Loop* Permodelan

Analisis untuk meningkatkan *augmentasi* stabilitas dilakukan meningkatkan performa dan membuat respons pesawat lebih cepat dan lebih teredam sesuai *poles* yang diinginkan. Dipilih *set poles closed loop* yang diinginkan (*desired poles*) sebagai berikut.

$$p = [-8, -1.2567 \pm 1.5738i, -1]$$

Set *poles* baru ini dirancang untuk memiliki rasio redaman *dutch roll* yang tetap dan memindahkan mode *roll* dan *spiral* ke lokasi yang lebih responsif (lebih jauh ke kiri di bidang-s). Menggunakan simulasi MATLAB, matrik K yang diperlukan untuk memindahkan poles ke lokasi baru didapatkan sebagai berikut.

$$K = \begin{bmatrix} 4.7496 & 2.1979 & 0.0995 & 0.5441 \\ 4.4814 & 1.3901 & -0.7487 & 0.9777 \end{bmatrix}$$

Nilai *gain* ini kemudian menjadi target atau *constraint* dari metode optimisasi yang dilakukan.

3.6 Optimasi *Gain* Belok Terkoordinasi

Tahap optimisasi *gain* (K) sistem kendali digunakan pendekatan optimisasi hibrida yang diimplementasikan dalam simulasi MATLAB. Berbeda dengan penyetelan *gain* konvensional, metode ini mengintegrasikan algoritma pencarian numerik *Quasi Newton* (BFGS) dengan validasi kestabilan berbasis *Linear Matrix Inequality* (LMI). Struktur ini dirancang untuk menyelesaikan masalah *non-convex*

dalam mencari parameter *gain* optimal $K\rho$ (laju *roll*), $K\beta$ (sudut *sideslip*), $K\dot{\beta}$ (laju *sideslip*), dan Kr (laju *yaw*) yang memenuhi kriteria meminimalkan deviasi pergerakan pada sudut *sideslip* (β) dan *percepatan lateral* (a_l) sekaligus menjamin *robustness* pesawat.

3.6.1 Formulasi Fungsi Biaya

Target optimasi performa belok terkoordinasi yang ingin dicapai diformulasikan dalam fungsi biaya skalar (J) yang dihitung berdasarkan respon waktu (*time-domain response*) pesawat terhadap gangguan turbulensi model Dryden. Fungsi biaya ini merupakan penjumlahan terbobot dari nilai *Root Mean Square* (RMS) empat variabel yaitu sebagai berikut.

$$J(K) = \omega_1 \text{rms}(\beta)^2 + \omega_2 \text{rms}(a_l)^2 + \omega_3 \text{rms}(\delta_r)^2 + \omega_4 \text{rms}(\dot{\delta}_r)^2 \quad (3.16)$$

Pembobotan (ω) ditentukan untuk memprioritaskan target yang ingin di capai meminimalkan deviasi pergerakan pada sudut *sideslip* (β), percepatan lateral (a_l), dan efisiensi penggunaan aktuator, dengan nilai parameter sebagai berikut.

$\omega_1 = 10.0$ adalah penalti deviasi sudut *sideslip* (β)

$\omega_2 = 15.0$ adalah penalti percepatan lateral (a_l)

$\omega_3 = 5.0$ adalah penalti defleksi rudder (δ_r)

$\omega_4 = 3.0$ adalah penalti laju defleksi rudder ($\dot{\delta}_r$)

3.6.2 Batas Variabel Keputusan

Variabel keputusan dalam optimisasi dibatasi (*constrained optimization*) untuk memastikan *gain* yang dihasilkan tetap berada dalam jangkauan operasional fisik aktuator pesawat CN235-220. Batasan yang diterapkan dalam program simulasi adalah sebagai berikut.

Gain sudut *sideslip* ($K\beta$) dibatasi pada rentang $[0, 12]$.

Gain laju *yaw* (Kr) dibatasi pada rentang $[0, 6]$.

Proses optimisasi dijalankan selama maksimum 50 iterasi atau hingga gradien fungsi biaya konvergen di bawah toleransi 1×10^{-4} .

3.7 Parameter Inisialisasi Simulasi

Uraian parameter-parameter utama yang ditentukan pada awal simulasi belok terkoordinasi mode *half bank* pesawat. Nilai-nilai ini digunakan untuk menafsirkan keluaran program karena menentukan kondisi penerbangan, model pesawat, pengaturan kontroler, dan lingkungan simulasi.

Tabel 3.1 Parameter Konstanta dan Konversi Simulasi Mode *Half Bank*

Kategori	Nama Parameter	Nilai	Unit / Deskripsi
Konstanta dan Konversi	GRAVITY	9.81	m/s ²
	FT_TO_M	0.3048	Faktor konversi kaki ke meter
	KNOTS_TO_MS	0.514444	Faktor konversi knot ke m/s
	DEG_TO_RAD	Pi/180	Faktor konversi derajat ke radian

Tabel 3.2 Parameter Kondisi Penerbangan Simulasi Mode *Half Bank*

Kategori	Nama Parameter	Nilai	Unit / Deskripsi
Kondisi Penerbangan	Altitude_ft	10000	ft
	Airspeed_knots	128	KTAS (Knots True Airspeed)
	turbulence_intensity	'moderate'	turbulence_intensity 'moderate' Mengatur intensitas hembusan ('ringan', 'sedang', 'berat')
	A_lateral	[-1.5, 0.12, 0, -0.9; 0, -0.3, 1, 0.05; -8.0, 0, -3.0, 1.0; 3.0, 0, 0, -0.8]	Matriks keadaan dinamika lateral pesawat
	B_lateral	[1.2, 0.1; 0, 0.05; 0.1, 1.5; 0.05, 0.8]	Matriks masukan dinamika lateral pesawat

Tabel 3.3 Parameter Gain Kontroler Dasar Simulasi Mode *Half Bank*

Kategori	Nama Parameter	Nilai	Unit / Deskripsi
Gain Kontroler Dasar	K_p_baseline	0.7	Gain proporsional untuk laju guling
	K_beta_baseline	10.0	Gain untuk sudut <i>sideslip</i>
	K_beta_dot_baseline	1.8	Gain untuk laju <i>sideslip</i>
	K_r_baseline	4.5	Gain untuk laju <i>yaw</i>
	K_I_baseline	0.5	Gain integral untuk <i>sideslip</i>
	K_rho_baseline	0.9	Gain untuk filter washout

Tabel 3.4 Parameter Desain dan Batasan LMI Simulasi Mode *Half Bank*

Kategori	Nama Parameter	H ₂ murni	H _∞ Murni	H ₂ /H _∞ campuran
Parameter Sistem	$q_{\beta}(H_2)$	6.0	-	4.0
	$r_{rudder}(H_2)$	5/0	-	6.0
	$q_{\beta}(H_{\infty})$	-	22.0	10.0
	$r_{control}(H_{\infty})$	-	2.5	3.0
	$\gamma(Batas\ H_{\infty})$	-	3.5	3.0
	Min_decay	0.12	0.05	0.08

Tabel 3.5 Parameter Sistem Simulasi Mode *Half Bank*

Kategori	Nama Parameter	Nilai	Unit / Deskripsi
Parameter Sistem	actuator_bandwidth_aileron	10	Rad/s
	actuator_bandwidth_rudder	8	Rad/s
	max_aileron_deg	20	Deg
	max_rudder_deg	20	Deg
	turn_coordination_tau	3.8	s (Konstanta waktu untuk filter washout)

Tabel 3.6 Parameter Pengaturan Simulasi Mode *Half Bank*

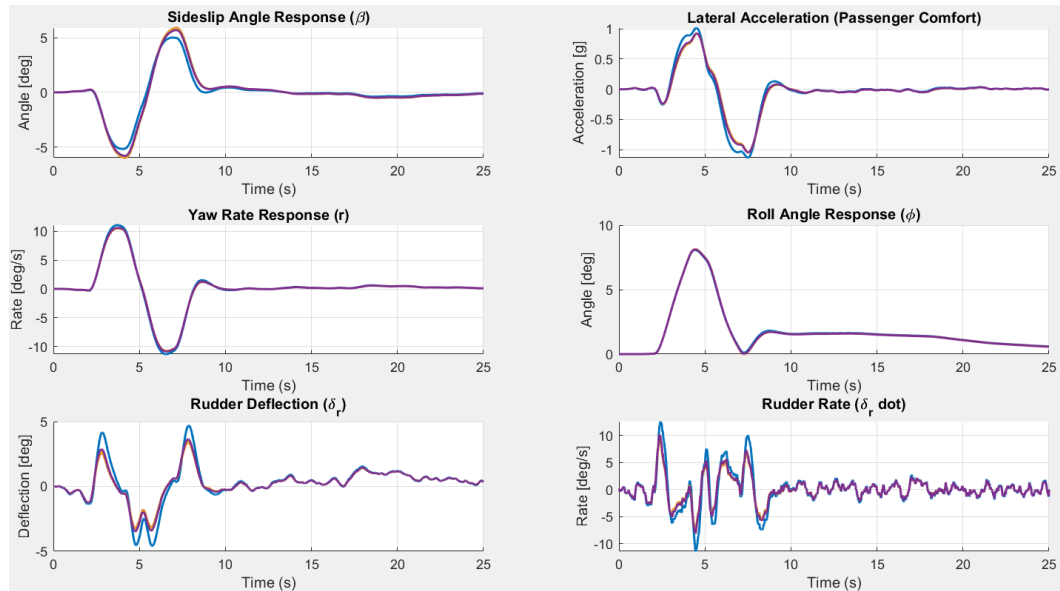
Kategori	Nama Parameter	Nilai	Unit / Deskripsi
Pengaturan Simulasi	simulation_time	25	s
	time_step	0.01	s

3.8 Pembahasan

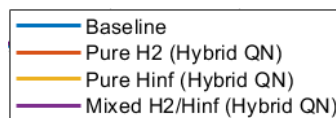
Simulasi belok terkoordinasi mode half-bank pesawat dilakukan menggunakan software Matlab didapatkan hasil performa dinamika lateral pesawat sebagai berikut.

Tabel 3.7 Matriks Performa Simulasi Dinamika lateral Pesawat

Matric	Baseline	H ₂ murni	H _∞ Murni	H ₂ /H _∞ campuran
Sideslip RMS (deg)	1.8023	2.0834	2.0680	2.0228
Sideslip Peak (deg)	5.1742	5.9736	5.9318	5.8085
Lat. Accel. RMS (g)	0.3634	0.3213	0.3223	0.3249
Rudder RMS (deg)	1.4940	1.1052	1.1232	1.1775
Rudder Rate RMS (deg/s)	3.0897	2.1612	2.1985	2.3124
H2 Norm	-	0.0302	-	0.0279
H-inf Gamma	-	-	3.5000	3.0000



Gambar 3.9 Grafik Performa Simulasi Dinamika lateral Pesawat



Gambar 3.10 Legenda Grafik Performa Simulasi Dinamika lateral Pesawat

Analisis kuantitatif berdasarkan data simulasi secara konklusif memvalidasi bahwa ketiga strategi optimisasi yang diusulkan H_2 murni, H_∞ murni, dan campuran H_2/H_∞ menunjukkan keunggulan performa dibandingkan dengan *kontroler Baseline*. Pendekatan optimisasi *hybrid* berbasis *Linear Matrix Inequality (LMI)* dan *Quasi-Newton* terbukti berhasil mencapai tujuan perancangan, dengan keunggulan utama teridentifikasi pada aspek kenyamanan penerbangan dan efisiensi aktuator. Peningkatan kenyamanan dibuktikan oleh reduksi metrik percepatan lateral (*Percepatan lateral RMS*) sebesar 11–12%, yang secara langsung mengurangi getaran yang dirasakan penumpang saat bermanuver. Secara simultan, efisiensi penggunaan aktuator meningkat drastis, ditandai oleh penurunan *Rudder Rate RMS* hingga 25% pada *kontroler* H_2/H_∞ . Data visual mengonfirmasi bahwa *kontroler* yang dioptimalkan menghasilkan gerakan kemudi (*rudder*) yang jauh lebih halus dan terkontrol, mengurangi beban mekanis pada aktuator sekaligus berkontribusi pada stabilitas manuver.

Teridentifikasi adanya sebuah *trade-off* pada performa *sideslip*. *Kontroler Baseline* menunjukkan nilai *Sideslip RMS* dan *Sideslip Peak* yang sedikit lebih rendah disebabkan oleh sifat kontrolnya yang agresif. Kontrol agresif ini mampu menekan *sideslip* secara instan, Akan tetapi dengan mengorbankan kenyamanan penerbangan dan efisiensi aktuator. Sebaliknya, *kontroler* yang dioptimalkan "mengizinkan" deviasi *sideslip* sesaat yang sedikit lebih besar Untuk mencapai manuver belok yang secara keseluruhan jauh lebih mulus dan efisien. Fenomena ini membuktikan keberhasilan pendekatan optimisasi multi objektif di mana performa ideal adalah keseimbangan antar variabel, bukan pencapaian ekstrem pada satu metrik saja.

Antara ketiga strategi yang diuji, kontroler campuran H_2/H_∞ menunjukkan keseimbangan terbaik, karena berhasil menyamai performa kenyamanan dari *kontroler* H_2 murni sambil tetap mempertahankan jaminan ketahanan (*robustness*) terhadap gangguan, yang didefinisikan oleh batas H -infinity (γ) sebesar 3.0000. Dengan demikian, hasil ini secara meyakinkan memvalidasi efektivitas metodologi optimisasi *hybrid LMI* dan *Quasi-Newton* dalam merancang sistem kendali yang unggul, serta menunjukkan potensi besar untuk implementasi praktis guna meningkatkan performa sistem *autopilot* pada pesawat CN235-220.

3.9 Diskusi

Keberhasilan perancangan kontroler campuran H_2/H_∞ memberikan dampak penting bagi pengembangan sistem autopilot di PT Dirgantara Indonesia. Kontroler ini mampu menyeimbangkan dua tujuan utama, yaitu performa nominal (tujuan H_2) dan ketahanan terhadap ketidakpastian model (*robustness*, tujuan H_∞). Dibandingkan dengan metode penyetelan gain klasik, pendekatan ini memberikan jaminan stabilitas matematis yang lebih baik terhadap gangguan dan variasi kondisi yang tidak terduga. Nilai tambah lainnya muncul dari penerapan metode optimisasi hibrida yang menggabungkan *Linear Matrix Inequality* (LMI) dengan algoritma quasi-Newton. LMI menyediakan dasar stabilitas yang kuat, sementara optimisasi pada *outer loop* melalui pendekatan quasi Newton memungkinkan penyetelan diarahkan pada aspek operasional tingkat tinggi, seperti kenyamanan penerbangan. Pendekatan bertingkat ini terbukti efektif dalam memperoleh gain kontroler yang stabil, robust, dan tetap sesuai dengan tujuan fungsional sistem.

Meskipun hasil simulasi cukup menjanjikan, studi ini masih terbatas pada satu kondisi penerbangan, sehingga kontroler perlu divalidasi pada berbagai ketinggian, kecepatan, dan konfigurasi berat pesawat. Langkah lanjutan yang disarankan adalah menerapkan *gain scheduling* dan melakukan pengujian *Hardware-in-the-Loop* (HIL) untuk memastikan kinerja kontroler pada perangkat keras autopilot sebelum pengujian terbang.

BAB IV

PENUTUP

4.1 Kesimpulan

Kesimpulan yang dapat diambil dari pelaksanaan Kerja Praktik ini adalah sebagai berikut.

1. Telah berhasil dirancang dan disimulasikan sistem kendali *robust* H_2/H_∞ untuk sistem belok terkoordinasi *mode half-bank* pada sistem *autopilot* pesawat CN235-220 menggunakan metode optimisasi *hybrid* yang menggabungkan *Linear Matrix Inequality* (LMI) dan algoritma *quasi-Newton*.
2. Ketiga strategi kendali yang dioptimalkan (H_2 murni, H_∞ murni, dan H_2/H_∞ campuran) didapati unggul dibandingkan dengan kontroler *Baseline*, dengan peningkatan signifikan pada kenyamanan penerbangan dirasakan penumpang yang ditandai oleh penurunan percepatan lateral RMS sebesar 11-12%.
3. Sistem kendali yang dioptimalkan secara drastis meningkatkan efisiensi aktuator, terbukti dari penurunan signifikan pada metrik *Rudder RMS* dan *Rudder Rate RMS* (hingga 25% pada H_2/H_∞ campuran), yang mengindikasikan gerakan bidang kendali yang lebih efektif dan potensi umur aktuator yang lebih panjang.
4. Kontroler H_2/H_∞ campuran menunjukkan keseimbangan terbaik antara performa dan ketahanan (*robustness*), berhasil menekan percepatan lateral dan penggunaan aktuator sekaligus menjamin stabilitas sistem terhadap ketidakpastian gangguan eksternal terburuk (*worst-case*) dengan nilai *sideslip* yang masih tergolong normal.
5. Teridentifikasi adanya *trade-off* yang terukur di mana kontroler yang dioptimalkan "mengizinkan" sedikit *sideslip* yang lebih besar untuk mencapai manuver belok yang lebih nyaman dan efisien secara keseluruhan, berbeda dengan kontroler *Baseline* yang menekan *sideslip* secara agresif dengan mengorbankan kenyamanan dan efisiensi aktuator.

4.2 Saran

Saran yang didapat dari pelaksanaan Kerja Praktik ini adalah sebagai berikut.

1. Perlu dilakukan validasi lebih lanjut terhadap performa kontroler yang telah dirancang pada berbagai kondisi penerbangan yang berbeda, seperti variasi ketinggian, kecepatan, dan konfigurasi berat pesawat, karena studi ini hanya berfokus pada satu titik kondisi operasi.
2. Disarankan untuk mengembangkan strategi *gain scheduling* guna mengadaptasi *gain* kontroler secara dinamis terhadap perubahan kondisi penerbangan, sehingga menjamin performa optimal di seluruh rentang operasi pesawat.
3. Sebagai langkah menuju implementasi nyata, direkomendasikan untuk melakukan pengujian lanjutan melalui simulasi *Hardware-in-the-Loop* (HIL) untuk mengevaluasi interaksi antara algoritma kendali dengan perangkat keras *autopilot* yang sesungguhnya sebelum dilakukan uji terbang.
4. Metodologi optimisasi *hybrid* LMI dan *quasi-Newton* yang terbukti efektif dalam studi ini dapat dieksplorasi untuk optimisasi sistem kendali lain pada pesawat CN235-220 atau proyek pengembangan pesawat lainnya di PT Dirgantara Indonesia.

DAFTAR PUSTAKA

- PT Dirgantara Indonesia (n.d.) Our Portfolio. Indonesian Aerospace. Available at: <https://www.indonesian-aerospace.com/en/>.
- PT Dirgantara Indonesia (n.d.) Corporate Overview. Indonesian Aerospace. Available at: https://www.indonesian-aerospace.com/en/about_us/corporate_overview.
- PT Dirgantara Indonesia (n.d.) CN235-220. Indonesian Aerospace. Available at: https://www.indonesian-aerospace.com/id/portofolio/pesawat_terbang/sayap_tetap/detil/1/cn235-220.
- PT Dirgantara Indonesia (n.d.) Autopilot AP5000R for CN235-220: Technical Description. Indonesian Aerospace.
- PT Dirgantara Indonesia (n.d.) Half Bank Mode Description. Indonesian Aerospace.
- PT Dirgantara Indonesia (n.d.) CN235-220: Technical Document CN235-220. Indonesian Aerospace.
- Chandrasekharan, P.C. (1996) Robust control of linear dynamical systems. San Diego: Academic Press.
- Scherer, C. & Weiland, S. (2015) *Linear Matrix Inequalities in Control*. Lecture Notes. Stuttgart & Eindhoven: University of Stuttgart; Eindhoven University of Technology.
- Shinners, S. M. (1998) *Advanced modern control system theory and design*. 2nd ed. New York: Wiley.
- Lewis, F.L. (1986) Optimal estimation with an introduction to stochastic control theory. New York: John Wiley & Sons.
- Popov, A. (2005) Less Conservative Mixed H_2/H_∞ Controller Design Using Multi-Objective Optimization. Laporan Proyek, Report 2005.15. Hamburg: Technische Universität Hamburg-Harburg

LAMPIRAN

Dokumentasi Pelaksanaan Kerja Praktik



Source Code Program MATLAB

```
function aircraft_analysis_comfort_focused()
    close all; clear; clc;

    params = initialize_aircraft_parameters();
    validate_parameters(params);
    display_flight_conditions(params);

    fprintf('Synthesizing controllers...\n');
    controllers = struct('name', {}, 'results', {});

    fprintf('\n--- Defining Baseline Controller ---\n');
    baseline_results = initialize_controller_results();
    baseline_results.method = 'Baseline';
    baseline_results.synthesis_success = true;
    baseline_results.K_beta = params.K_beta_baseline;
    baseline_results.K_r = params.K_r_baseline;
    controllers(1).name = 'Baseline';
    controllers(1).results = baseline_results;
    fprintf(' Baseline controller defined (K_beta: %.2f, K_r: %.2f).\n', ...
        baseline_results.K_beta, baseline_results.K_r);

    synthesis_functions = {
        @synthesize_pure_h2_hybrid_qn, 'Pure H2 (Hybrid QN)';
        @synthesize_pure_hinf_hybrid_qn, 'Pure Hinf (Hybrid QN)';
        @synthesize_mixed_h2_hinf_hybrid_qn, 'Mixed H2/Hinf (Hybrid QN)'
    };

    for i = 1:size(synthesis_functions, 1)
        synth_func = synthesis_functions{i, 1};
        base_name = synthesis_functions{i, 2};
        fprintf('\n--- Synthesizing %s Controller (Comfort Focused) ---\n', base_name);
        lmi_results = synth_func(params);
        controllers(end+1).name = base_name;
        controllers(end).results = lmi_results;
    end
end
```

```

fprintf('\nSimulating responses for successful controllers...\n');
simulation_results = {};
[gust_series, gust_params] = generate_dryden_gust_series(params);

for i = 1:length(controllers)
    ctrl = controllers(i);
    if isfield(ctrl.results, 'synthesis_success') && ctrl.results.synthesis_success
        fprintf(' Simulating %s controller...\n', ctrl.name);
        [time, states, controls] = simulate_2state_optimized_response(params, ctrl.results,
gust_series);
        sim_result = struct();
        sim_result.time = time;
        sim_result.states = states;
        sim_result.controls = controls;
        sim_result.name = ctrl.name;
        sim_result.results = ctrl.results;
        simulation_results(end+1) = sim_result;
    else
        fprintf(' Skipping simulation for %s (synthesis failed).\n', ctrl.name);
    end
end

if isempty(simulation_results)
    fprintf('\nNo controllers were successfully synthesized and simulated. Skipping analysis.\n');
    return;
end

fprintf('Generating performance comparison...\n');
generate_performance_comparison(simulation_results, params, gust_series, gust_params);
print_performance_metrics(simulation_results, params);
fprintf('\nAircraft controller analysis complete.\n');
end

function controller_results = synthesize_pure_h2_hybrid_qn(params)
    controller_results = initialize_controller_results();
    if ~exist('sdpvar', 'file')
        fprintf(' YALMIP not available. Using baseline gains.\n');
        controller_results.synthesis_success = true;
        controller_results.method = 'Pure H2 (Hybrid QN)';
        controller_results.K_beta = params.K_beta_baseline;
        controller_results.K_r = params.K_r_baseline;
        return;
    end

    fprintf(' Starting Pure H2 (Hybrid QN) Optimization...\n');
    max_iterations = 30;
    tolerance = 1e-4;
    step_size = 0.1;
    K_current = [params.K_beta_baseline; params.K_r_baseline];
    cost_history = [];
    [gust_series, ~] = generate_dryden_gust_series(params);
    fprintf(' Initial gains: K_beta=%.3f, K_r=%.3f\n', K_current(1), K_current(2));
    [A_2state, B_2state] = get_2state_model(params);
    nx = 2; nu = 1;
    q_beta = 6.0; q_r = 2.5; r_rudder = 5.0; min_decay = 0.12;
    Q = diag([q_beta, q_r]);
    R = r_rudder;
    B2 = [0.05; 0.02];
    C2 = [sqrt(q_beta), 0; 0, sqrt(q_r); 0, 0];

```

```

D2u = [0; 0; sqrt(r_rudder)];
eps_P = 1e-4;
options = sdpsettings('verbose', 0, 'solver', 'mosek');

for iter = 1:max_iterations
    fprintf('    Iter %2d: ', iter);
    P = sdpvar(nx, nx, 'symmetric');
    Y = sdpvar(nu, nx, 'full');
    W = sdpvar(size(C2,1), size(C2,1), 'symmetric');
    K_check = [K_current(1), K_current(2)];
    A_cl_check = A_2state - B_2state * K_check;
    constraints_h2 = [P >= eps_P*eye(nx), W >= 1e-6*eye(size(W,1)), P <= 100*eye(nx)];
    constraints_h2 = [constraints_h2, Y(1,1) >= 0, Y(1,1) <= 15.0];
    constraints_h2 = [constraints_h2, Y(1,2) >= 0, Y(1,2) <= 6.0];
    LMI_stability_h2 = A_cl_check*P + P*A_cl_check' + B_2state*Y + Y'*B_2state' + 2*min_decay*P;
    constraints_h2 = [constraints_h2, LMI_stability_h2 <= -eps_P*eye(nx)];
    LMI_h2_lyap = [A_cl_check*P + P*A_cl_check' + B_2state*Y + Y'*B_2state', B2; B2', -
eye(size(B2,2))];
    constraints_h2 = [constraints_h2, LMI_h2_lyap <= -eps_P*eye(size(LMI_h2_lyap,1))];
    LMI_h2_perf = [W, (C2*P + D2u*Y); (C2*P + D2u*Y)', P];
    constraints_h2 = [constraints_h2, LMI_h2_perf >= eps_P*eye(size(LMI_h2_perf,1))];
    sol_lmi = optimize(constraints_h2, [], options);
    lmi_feasible = (sol_lmi.problem == 0);

    if ~lmi_feasible
        fprintf('LMI infeasible, reducing step size...\n');
        step_size = step_size * 0.5;
        if step_size < 1e-6
            fprintf('    Optimization failed: Cannot find feasible region\n');
            break;
        end
        continue;
    end

    temp_controller = struct('synthesis_success', true, 'K_beta', K_current(1), 'K_r',
K_current(2));
    [~, states, controls] = simulate_2state_optimized_response(params, temp_controller,
gust_series);
    J_current = compute_comfort_cost_function(states, controls, params);
    cost_history(end+1) = J_current;
    fprintf('K_beta=%.3f, K_r=%.3f, Cost=%.4f', K_current(1), K_current(2), J_current);
    gradient = compute_numerical_gradient(params, K_current, gust_series, J_current);

    if iter == 1
        H_inv = eye(2);
    else
        H_inv = update_inverse_hessian(H_inv, K_current - K_prev, gradient - grad_prev);
    end

    K_prev = K_current;
    grad_prev = gradient;
    K_new = K_current - step_size * H_inv * gradient;
    K_new(1) = max(0, min(12, K_new(1)));
    K_new(2) = max(0, min(6, K_new(2)));

    if norm(K_new - K_current) < tolerance
        fprintf(' -> CONVERGED\n');
        K_current = K_new;
        break;

```

```

end

K_current = K_new;
fprintf(' -> Updated\n');
end

P = sdpvar(nx, nx, 'symmetric');
Y = sdpvar(nu, nx, 'full');
W = sdpvar(size(C2,1), size(C2,1), 'symmetric');
K_final = [K_current(1), K_current(2)];
A_cl_final = A_2state - B_2state * K_final;
constraints_h2_final = [P >= eps_P*eye(nx), W >= 1e-6*eye(size(W,1)), P <= 100*eye(nx)];
constraints_h2_final = [constraints_h2_final, Y(1,1) >= 0, Y(1,1) <= 15.0];
constraints_h2_final = [constraints_h2_final, Y(1,2) >= 0, Y(1,2) <= 6.0];
LMI_stability_h2_final = A_cl_final*P + P*A_cl_final' + B_2state*Y + Y'*B_2state' + 2*min_decay*P;
constraints_h2_final = [constraints_h2_final, LMI_stability_h2_final <= -eps_P*eye(nx)];
LMI_h2_lyap_final = [A_cl_final*P + P*A_cl_final' + B_2state*Y + Y'*B_2state', B2; B2', -
eye(size(B2,2))];
constraints_h2_final = [constraints_h2_final, LMI_h2_lyap_final <= -
eps_P*eye(size(LMI_h2_lyap_final,1))];
LMI_h2_perf_final = [W, (C2*P + D2u*Y); (C2*P + D2u*Y)', P];
constraints_h2_final = [constraints_h2_final, LMI_h2_perf_final >=
eps_P*eye(size(LMI_h2_perf_final,1))];
sol_final = optimize(constraints_h2_final, trace(W), options);

controller_results.synthesis_success = (sol_final.problem == 0);
controller_results.method = 'Pure H2 (Hybrid QN)';
controller_results.K_beta = K_current(1);
controller_results.K_r = K_current(2);
controller_results.optimization_iterations = length(cost_history);
controller_results.final_cost = cost_history(end);
controller_results.cost_history = cost_history;
controller_results.closed_loop_poles = eig(A_cl_final);
if controller_results.synthesis_success, controller_results.h2_norm = value(trace(W)); else,
controller_results.h2_norm = NaN; end

fprintf('    FINAL: K_beta=%.3f, K_r=%.3f, Cost=%.4f, Iterations=%d\n', ...
        K_current(1), K_current(2), cost_history(end), length(cost_history));
end

function controller_results = synthesize_pure_hinf_hybrid_qn(params)
controller_results = initialize_controller_results();
if ~exist('sdpvar', 'file')
    fprintf(' YALMIP not available. Using baseline gains.\n');
    controller_results.synthesis_success = true;
    controller_results.method = 'Pure Hinf (Hybrid QN)';
    controller_results.K_beta = params.K_beta_baseline;
    controller_results.K_r = params.K_r_baseline;
    return;
end

fprintf(' Starting Pure Hinf (Hybrid QN) Optimization...\n');
max_iterations = 30;
tolerance = 1e-4;
step_size = 0.1;
K_current = [params.K_beta_baseline; params.K_r_baseline];
cost_history = [];
[gust_series, ~] = generate_dryden_gust_series(params);
fprintf('    Initial gains: K_beta=%.3f, K_r=%.3f\n', K_current(1), K_current(2));

```



```

[A_2state, B_2state] = get_2state_model(params);
nx = 2; nu = 1;
gamma_target = 3.5; q_beta = 22.0; q_r = 1.5; r_control = 2.5; min_decay = 0.05;
B1 = [0.1; 0.05];
C1 = [sqrt(q_beta), 0; 0, sqrt(q_r); 0, 0];
D12 = [0; 0; sqrt(r_control)];
eps_P = 1e-4;
options = sdpsettings('verbose', 0, 'solver', 'mosek');

for iter = 1:max_iterations
    fprintf('    Iter %2d: ', iter);
    P = sdpvar(nx, nx, 'symmetric');
    Y = sdpvar(nu, nx, 'full');
    K_check = [K_current(1), K_current(2)];
    A_cl_check = A_2state - B_2state * K_check;
    constraints_hinf = [P >= eps_P*eye(nx), P <= 50*eye(nx)];
    constraints_hinf = [constraints_hinf, Y(1,1) >= 0, Y(1,1) <= 15.0];
    constraints_hinf = [constraints_hinf, Y(1,2) >= 0, Y(1,2) <= 6.0];
    LMI_hinf_check = [A_cl_check*P + P*A_cl_check' + B_2state*Y + Y'*B_2state' + 2*min_decay*P, B1,
    (C1*P + D12*Y)'];
    B1', -gamma_target*eye(size(B1,2)), zeros(size(B1,2), size(C1,1));
    C1*P + D12*Y, zeros(size(C1,1), size(B1,2)), -gamma_target*eye(size(C1,1))];
    constraints_hinf = [constraints_hinf, LMI_hinf_check <= -eps_P*eye(size(LMI_hinf_check,1))];
    sol_lmi = optimize(constraints_hinf, [], options);
    lmi_feasible = (sol_lmi.problem == 0);

    if ~lmi_feasible
        fprintf('LMI infeasible, reducing step size...\n');
        step_size = step_size * 0.5;
        if step_size < 1e-6
            fprintf('    Optimization failed: Cannot find feasible region\n');
            break;
        end
        continue;
    end

    temp_controller = struct('synthesis_success', true, 'K_beta', K_current(1), 'K_r',
    K_current(2));
    [~, states, controls] = simulate_2state_optimized_response(params, temp_controller,
    gust_series);
    J_current = compute_comfort_cost_function(states, controls, params);
    cost_history(end+1) = J_current;
    fprintf('K_beta=%.3f, K_r=%.3f, Cost=%.4f', K_current(1), K_current(2), J_current);
    gradient = compute_numerical_gradient(params, K_current, gust_series, J_current);

    if iter == 1
        H_inv = eye(2);
    else
        H_inv = update_inverse_hessian(H_inv, K_current - K_prev, gradient - grad_prev);
    end

    K_prev = K_current;
    grad_prev = gradient;
    K_new = K_current - step_size * H_inv * gradient;
    K_new(1) = max(0, min(12, K_new(1)));
    K_new(2) = max(0, min(6, K_new(2)));

    if norm(K_new - K_current) < tolerance
        fprintf(' -> CONVERGED\n');

```

```

        K_current = K_new;
        break;
    end

    K_current = K_new;
    fprintf(' -> Updated\n');
end

P = sdpvar(nx, nx, 'symmetric');
Y = sdpvar(nu, nx, 'full');
K_final = [K_current(1), K_current(2)];
A_cl_final = A_2state - B_2state * K_final;
constraints_hinf_final = [P >= eps_P*eye(nx), P <= 50*eye(nx)];
constraints_hinf_final = [constraints_hinf_final, Y(1,1) >= 0, Y(1,1) <= 15.0];
constraints_hinf_final = [constraints_hinf_final, Y(1,2) >= 0, Y(1,2) <= 6.0];
LMI_hinf_final = [A_cl_final*P + P*A_cl_final' + B_2state*Y + Y'*B_2state' + 2*min_decay*P, B1,
(C1*P + D12*Y)'];

B1', -gamma_target*eye(size(B1,2)), zeros(size(B1,2), size(C1,1));
C1*P + D12*Y, zeros(size(C1,1), size(B1,2)), -gamma_target*eye(size(C1,1))];
constraints_hinf_final = [constraints_hinf_final, LMI_hinf_final <= -
eps_P*eye(size(LMI_hinf_final,1))];
sol_final = optimize(constraints_hinf_final, [], options);

controller_results.synthesis_success = (sol_final.problem == 0);
controller_results.method = 'Pure Hinf (Hybrid QN)';
controller_results.K_beta = K_current(1);
controller_results.K_r = K_current(2);
controller_results.optimization_iterations = length(cost_history);
controller_results.final_cost = cost_history(end);
controller_results.cost_history = cost_history;
controller_results.closed_loop_poles = eig(A_cl_final);
if controller_results.synthesis_success, controller_results.gamma_inf = gamma_target; else,
controller_results.gamma_inf = NaN; end

fprintf('    FINAL: K_beta=%.3f, K_r=%.3f, Cost=%.4f, Iterations=%d\n', ...
        K_current(1), K_current(2), cost_history(end), length(cost_history));
end

function controller_results = synthesize_mixed_h2_hinf_hybrid_qn(params)
    controller_results = initialize_controller_results();
    if ~exist('sdpvar', 'file')
        fprintf(' YALMIP not available. Using baseline gains.\n');
        controller_results.synthesis_success = true;
        controller_results.method = 'Mixed H2/Hinf (Hybrid QN)';
        controller_results.K_beta = params.K_beta_baseline;
        controller_results.K_r = params.K_r_baseline;
        return;
    end

    fprintf(' Starting Mixed H2/Hinf (Hybrid QN) Optimization...\n');
    max_iterations = 30;
    tolerance = 1e-4;
    step_size = 0.1;
    K_current = [params.K_beta_baseline; params.K_r_baseline];
    cost_history = [];
    [gust_series, ~] = generate_dryden_gust_series(params);
    fprintf('    Initial gains: K_beta=%.3f, K_r=%.3f\n', K_current(1), K_current(2));
    [A_2state, B_2state] = get_2state_model(params);
    nx = 2; nu = 1;

```

```

gamma_fixed = 3.0; q_h2_beta = 4.0; r_h2 = 6.0; q_inf_beta = 10.0; r_inf = 3.0; min_decay = 0.08;
B2_h2 = [0.08; 0.03]; B1_inf = [0.12; 0.06];
C2 = [sqrt(q_h2_beta), 0; 0, 0; 0, 0, 0]; D2u = [0; 0; sqrt(r_h2)];
C1 = [sqrt(q_inf_beta), 0; 0, 0; 0, 0]; D12 = [0; 0; sqrt(r_inf)];
eps_P = 1e-4;
options = sdpsettings('verbose', 0, 'solver', 'mosek');

for iter = 1:max_iterations
    fprintf('    Iter %2d: ', iter);
    P = sdpvar(nx, nx, 'symmetric');
    Y = sdpvar(nu, nx, 'full');
    W = sdpvar(size(C2,1), size(C2,1), 'symmetric');
    K_check = [K_current(1), K_current(2)];
    A_cl_check = A_2state - B_2state * K_check;
    constraints_mixed = [P >= eps_P*eye(nx), P <= 30*eye(nx), W >= 1e-6*eye(size(W,1))];
    constraints_mixed = [constraints_mixed, Y(1,1) >= -0.5, Y(1,1) <= 15.0];
    constraints_mixed = [constraints_mixed, Y(1,2) >= -0.5, Y(1,2) <= 6.0];
    LMI_hinf_check = [A_cl_check*P + P*A_cl_check' + B_2state*Y + Y'*B_2state' + 2*min_decay*P,
    B1_inf, (C1*P + D12*Y)'];
    size(B1_inf,2),
    size(C1,1));
    C1*P      +      D12*Y,      zeros(size(C1,1),      size(B1_inf,2)),      -
gamma_fixed*eye(size(C1,1));
    constraints_mixed = [constraints_mixed, LMI_hinf_check <= -eps_P*eye(size(LMI_hinf_check,1))];
    LMI_h2_lyap = [A_cl_check*P + P*A_cl_check' + B_2state*Y + Y'*B_2state', B2_h2; B2_h2', -
    eye(size(B2_h2,2))];
    constraints_mixed = [constraints_mixed, LMI_h2_lyap <= -eps_P*eye(size(LMI_h2_lyap,1))];
    LMI_h2_perf = [W, (C2*P + D2u*Y); (C2*P + D2u*Y)', P];
    constraints_mixed = [constraints_mixed, LMI_h2_perf >= eps_P*eye(size(LMI_h2_perf,1))];
    sol_lmi = optimize(constraints_mixed, [], options);
    lmi_feasible = (sol_lmi.problem == 0);

    if ~lmi_feasible
        fprintf('LMI infeasible, reducing step size...\n');
        step_size = step_size * 0.5;
        if step_size < 1e-6
            fprintf('    Optimization failed: Cannot find feasible region\n');
            break;
        end
        continue;
    end

    temp_controller = struct('synthesis_success', true, 'K_beta', K_current(1), 'K_r',
    K_current(2));
    [~, states, controls] = simulate_2state_optimized_response(params, temp_controller,
    gust_series);
    J_current = compute_comfort_cost_function(states, controls, params);
    cost_history(end+1) = J_current;
    fprintf('K_beta=%.3f, K_r=%.3f, Cost=%.4f', K_current(1), K_current(2), J_current);
    gradient = compute_numerical_gradient(params, K_current, gust_series, J_current);

    if iter == 1
        H_inv = eye(2);
    else
        H_inv = update_inverse_hessian(H_inv, K_current - K_prev, gradient - grad_prev);
    end

    K_prev = K_current;
    grad_prev = gradient;

```

```

        K_new = K_current - step_size * H_inv * gradient;
        K_new(1) = max(0, min(12, K_new(1)));
        K_new(2) = max(0, min(6, K_new(2)));

        if norm(K_new - K_current) < tolerance
            fprintf(' -> CONVERGED\n');
            K_current = K_new;
            break;
        end

        K_current = K_new;
        fprintf(' -> Updated\n');
    end

    P = sdpvar(nx, nx, 'symmetric');
    Y = sdpvar(nu, nx, 'full');
    W = sdpvar(size(C2,1), size(C2,1), 'symmetric');
    K_final = [K_current(1), K_current(2)];
    A_cl_final = A_2state - B_2state * K_final;
    constraints_mixed_final = [P >= eps_P*eye(nx), P <= 30*eye(nx), W >= 1e-6*eye(size(W,1))];
    constraints_mixed_final = [constraints_mixed_final, Y(1,1) >= -0.5, Y(1,1) <= 15.0];
    constraints_mixed_final = [constraints_mixed_final, Y(1,2) >= -0.5, Y(1,2) <= 6.0];
    LMI_hinf_final = [A_cl_final*P + P*A_cl_final' + B_2state*Y + Y'*B_2state' + 2*min_decay*P, B1_inf,
    (C1*P + D12*Y)'];
    B1_inf', -gamma_fixed*eye(size(B1_inf,2)), zeros(size(B1_inf,2), size(C1,1));
    C1*P + D12*Y, zeros(size(C1,1), size(B1_inf,2)), -gamma_fixed*eye(size(C1,1))];
    constraints_mixed_final = [constraints_mixed_final, LMI_hinf_final <= -
    eps_P*eye(size(LMI_hinf_final,1))];
    LMI_h2_lyap_final = [A_cl_final*P + P*A_cl_final' + B_2state*Y + Y'*B_2state', B2_h2; B2_h2', -
    eye(size(B2_h2,2))];
    constraints_mixed_final = [constraints_mixed_final, LMI_h2_lyap_final <= -
    eps_P*eye(size(LMI_h2_lyap_final,1))];
    LMI_h2_perf_final = [W, (C2*P + D2u*Y); (C2*P + D2u*Y)', P];
    constraints_mixed_final = [constraints_mixed_final, LMI_h2_perf_final >=
    eps_P*eye(size(LMI_h2_perf_final,1))];
    sol_final = optimize(constraints_mixed_final, trace(W), options);

    controller_results.synthesis_success = (sol_final.problem == 0);
    controller_results.method = 'Mixed H2/Hinf (Hybrid QN)';
    controller_results.K_beta = K_current(1);
    controller_results.K_r = K_current(2);
    controller_results.optimization_iterations = length(cost_history);
    controller_results.final_cost = cost_history(end);
    controller_results.cost_history = cost_history;
    controller_results.closed_loop_poles = eig(A_cl_final);
    if controller_results.synthesis_success
        controller_results.h2_norm = value(trace(W));
        controller_results.gamma_inf = gamma_fixed;
    else
        controller_results.h2_norm = NaN;
        controller_results.gamma_inf = NaN;
    end

    fprintf('    FINAL: K_beta=%.3f, K_r=%.3f, Cost=%.4f, Iterations=%d\n', ...
        K_current(1), K_current(2), cost_history(end), length(cost_history));
end

function [time, states, controls] = simulate_2state_optimized_response(params, controller_results,
gust_series)

```

```

time = 0:params.time_step:params.simulation_time;
N = length(time);
states = zeros(9, N);
controls = zeros(2, N);
K_beta_opt = controller_results.K_beta;
K_r_opt = controller_results.K_r;
K_p = params.K_p_baseline;
K_beta_dot = params.K_beta_dot_baseline;
K_I = params.K_I_baseline;
K_rho = params.K_rho_baseline;
a_ail = params.actuator_bandwidth_aileron;
a_rud = params.actuator_bandwidth_rudder;
max_aileron = params.max_aileron_deg * params.DEG_TO_RAD;
max_rudder = params.max_rudder_deg * params.DEG_TO_RAD;
fprintf('    Using optimized gains: K_beta=%.3f, K_r=%.3f\n', K_beta_opt, K_r_opt);

for k = 1:N-1
    p = states(1,k); beta = states(2,k); beta_dot = states(3,k); r = states(4,k);
    phi = states(5,k); eI = states(6,k); x_wof = states(7,k);
    delta_a = states(8,k); delta_r = states(9,k);

    aileron_feedback = -K_p * p;
    rudder_feedback = -K_beta_opt * beta - K_beta_dot * beta_dot - K_r_opt * r;
    rudder_integral = -K_I * eI;
    rudder_washout = -K_rho * x_wof;

    aileron_pilot_input = generate_aileron_command(time(k), params);
    delta_a_cmd = aileron_pilot_input + aileron_feedback;
    delta_r_cmd = rudder_feedback + rudder_integral + rudder_washout;

    delta_a_cmd = max(-max_aileron, min(max_aileron, delta_a_cmd));
    delta_r_cmd = max(-max_rudder, min(max_rudder, delta_r_cmd));
    controls(1,k) = delta_a_cmd;
    controls(2,k) = delta_r_cmd;

    delta_a_dot = -a_ail*delta_a + a_ail*delta_a_cmd;
    delta_r_dot = -a_rud*delta_r + a_rud*delta_r_cmd;

    B_gust = [0; 0; 0.1; 0];
    x_lateral = [p; beta; beta_dot; r];
    dx_lateral = params.A_lateral*x_lateral + params.B_lateral*[delta_a; delta_r] +
    B_gust*gust_series(k);

    phi_dot = p + r * tan(states(5,k));
    eI_dot = beta;
    x_wof_dot = -(1/params.turn_coordination_tau)*x_wof + r;

    state_derivatives = [dx_lateral; phi_dot; eI_dot; x_wof_dot; delta_a_dot; delta_r_dot];
    states(:,k+1) = rk4_integration(states(:,k), state_derivatives, params.time_step);
end
controls(:,N) = controls(:,N-1);
end

function print_performance_metrics(simulation_results, params)
    fprintf('\n===== Performance & Comfort Metrics (Comfort Focused LMI)
=====');
    header_format = '%-25s';
    fprintf(header_format, 'Metric');
    for i = 1:length(simulation_results), fprintf('%-30s', simulation_results{i}.name); end

```

```

fprintf('\n');
fprintf(repmat('-', 1, 25 + 30*length(simulation_results))); fprintf('\n');

metrics = {'Sideslip RMS (deg)', 'Sideslip Peak (deg)', 'Lat. Accel. RMS (g)', 'Rudder RMS (deg)',
'Rudder Rate RMS (deg/s)', 'K_beta (optimized)', 'K_r (optimized)', 'Optimization Iterations', 'Final
Cost Function', 'H2 Norm', 'H-inf Gamma'};

for m_idx = 1:length(metrics)
    fprintf(header_format, metrics{m_idx});
    for ctrl_idx = 1:length(simulation_results)
        res = simulation_results{ctrl_idx};
        val = NaN;

        switch metrics{m_idx}
            case 'Sideslip RMS (deg)'
                val = rms(res.states(2, :) / params.DEG_TO_RAD);
            case 'Sideslip Peak (deg)'
                val = max(abs(res.states(2, :) / params.DEG_TO_RAD));
            case 'Lat. Accel. RMS (g)'
                beta_dot_rad_s = res.states(3, :);
                r_rad_s = res.states(4, :);
                phi_rad = res.states(5, :);
                lat_accel_ms2 = params.airspeed_ms * (r_rad_s - params.GRAVITY / params.airspeed_ms
.* sin(phi_rad)) + params.airspeed_ms .* beta_dot_rad_s;
                val = rms(lat_accel_ms2 / params.GRAVITY);
            case 'Rudder RMS (deg)'
                val = rms(res.controls(2, :) / params.DEG_TO_RAD);
            case 'Rudder Rate RMS (deg/s)'
                rudder_deg = res.controls(2, :) / params.DEG_TO_RAD;
                rudder_rate_degs = diff(rudder_deg) / params.time_step;
                val = rms(rudder_rate_degs);
            case 'K_beta (optimized)'
                val = res.results.K_beta;
            case 'K_r (optimized)'
                val = res.results.K_r;
            case 'Optimization Iterations'
                if isfield(res.results, 'optimization_iterations'), val =
res.results.optimization_iterations; end
            case 'Final Cost Function'
                if isfield(res.results, 'final_cost'), val = res.results.final_cost; end
            case 'H2 Norm'
                val = res.results.h2_norm;
            case 'H-inf Gamma'
                val = res.results.gamma_inf;
        end

        if isnan(val), fprintf('%-30s', 'N/A'); else, fprintf('%-30.4f', val); end
    end
    fprintf('\n');
end
fprintf('\n');
end

function generate_performance_comparison(simulation_results, params, ~, ~)
    figure('Name', 'Aircraft Response Comparison (Comfort Focused LMI)', 'Position', [100, 50, 1200,
950], 'NumberTitle', 'off');
    plot_titles = {'Sideslip Angle Response (\beta)', 'Lateral Acceleration (Passenger Comfort)', 'Yaw
Rate Response (r)', 'Roll Angle Response (\phi)', 'Rudder Deflection (\delta_r)', 'Rudder Rate (\delta_r
dot)'};

```

```

        y_labels = {'Angle [deg]', 'Acceleration [g]', 'Rate [deg/s]', 'Angle [deg]', 'Deflection [deg]',
                    'Rate [deg/s]'};
        colors = lines(length(simulation_results));

    for i = 1:6
        subplot(3, 2, i);
        set(gca, 'FontSize', 11, 'FontName', 'Arial'); hold on; grid on;

        for ctrl_idx = 1:length(simulation_results)
            res = simulation_results(ctrl_idx);
            time = res.time;
            data = [];

            switch i
                case 1, data = res.states(2, :) / params.DEG_TO_RAD;
                case 2
                    beta_dot_rad_s = res.states(3, :);
                    r_rad_s = res.states(4, :);
                    phi_rad = res.states(5, :);
                    lat_accel_ms2 = params.airspeed_ms * (r_rad_s - params.GRAVITY / params.airspeed_ms
                    .* sin(phi_rad)) + params.airspeed_ms .* beta_dot_rad_s;
                    data = lat_accel_ms2 / params.GRAVITY;
                case 3, data = res.states(4, :) / params.DEG_TO_RAD;
                case 4, data = res.states(5, :) / params.DEG_TO_RAD;
                case 5, data = res.controls(2, :) / params.DEG_TO_RAD;
                case 6
                    rudder_deg = res.controls(2, :) / params.DEG_TO_RAD;
                    rudder_rate_degs = [0, diff(rudder_deg) / params.time_step];
                    data = rudder_rate_degs;
            end

            plot(time, data, 'LineWidth', 1.8, 'Color', colors(ctrl_idx,:));
        end

        xlabel('Time (s)'); ylabel(y_labels{i}); title(plot_titles{i});
        xlim([0 params.simulation_time]);
        if i == 1
            legend_names = cellfun(@(c) strrep(c.name, '_', ' '), simulation_results, 'UniformOutput',
            false);
            legend(legend_names, 'Location', 'northeast');
        end
    end

    sgtitle('Aircraft Response Comparison: Comfort-Focused LMI Optimization', 'FontSize', 16,
    'FontWeight', 'bold');

    if ~isempty(simulation_results)
        sim_names = cellfun(@(s) s.name, simulation_results, 'UniformOutput', false);
        if any(contains(sim_names, 'Hybrid QN'))
            figure('Name', 'Quasi-Newton Convergence', 'Position', [1300, 100, 400, 300]);
            qn_idx = find(contains(sim_names, 'Hybrid QN'), 1);
            if ~isempty(qn_idx) && isfield(simulation_results{qn_idx}.results, 'cost_history')
                plot(1:length(simulation_results{qn_idx}.results.cost_history), ...
                    simulation_results{qn_idx}.results.cost_history, 'b-o', 'LineWidth', 2);
                xlabel('Iteration'); ylabel('Cost Function');
                title('Quasi-Newton Convergence'); grid on;
            end
        end
    end
end
end
end

```

```

function params = initialize_aircraft_parameters()
    params.GRAVITY = 9.81; params.FT_TO_M = 0.3048;
    params.KNOTS_TO_MS = 0.514444; params.DEG_TO_RAD = pi/180;
    params.altitude_ft = 10000; params.airspeed_knots = 128;
    params.altitude_m = params.altitude_ft * params.FT_TO_M;
    params.airspeed_ms = params.airspeed_knots * params.KNOTS_TO_MS;
    [~, ~, params.density_kgm3] = calculate_atmospheric_properties(params.altitude_m);
    params.A_lateral = [-1.5, 0.12, 0, -0.9; 0, -0.3, 1, 0.05; -8.0, 0, -3.0, 1.0; 3.0, 0, 0, -0.8];
    params.B_lateral = [1.2, 0.1; 0, 0.05; 0.1, 1.5; 0.05, 0.8];
    params.K_p_baseline = 0.7;
    params.K_beta_baseline = 10.0;
    params.K_beta_dot_baseline = 1.8;
    params.K_r_baseline = 4.5;
    params.K_I_baseline = 0.5;
    params.K_rho_baseline = 0.9;
    params.actuator_bandwidth_aileron = 10; params.actuator_bandwidth_rudder = 8;
    params.max_aileron_deg = 20; params.max_rudder_deg = 20;
    params.turn_coordination_tau = 3.8;
    params.simulation_time = 25; params.time_step = 0.01;
    params.turbulence_intensity = 'moderate';
end

function [gust_series_mps, gust_params] = generate_dryden_gust_series(params)
    fprintf('\nGenerating Dryden turbulence model...\n');
    altitude_ft = params.altitude_ft; V = params.airspeed_ms;
    if altitude_ft < 1000, L_v = altitude_ft * params.FT_TO_M; else, L_v = 1000 * params.FT_TO_M; end
    switch lower(params.turbulence_intensity)
        case 'light', sigma_w = 1.5; case 'moderate', sigma_w = 3.0;
        case 'severe', sigma_w = 6.0; otherwise, sigma_w = 3.0;
    end
    gust_filter_tf = tf(sigma_w * sqrt(2*V/L_v), [1, V/L_v]);
    time_vec = 0:params.time_step:params.simulation_time;
    white_noise = randn(size(time_vec));
    gust_series_mps = lsim(gust_filter_tf, white_noise, time_vec);
    gust_params.sigma_v = sigma_w; gust_params.L_v = L_v;
    gust_params.intensity = params.turbulence_intensity;
    fprintf(' Intensity: '%s' (sigma_v = %.2f m/s), Scale Length: %.1f m\n', ...
        gust_params.intensity, gust_params.sigma_v, gust_params.L_v);
end

function [A_2state, B_2state] = get_2state_model(params)
    A_2state = [params.A_lateral(2,2), params.A_lateral(2,4);
               params.A_lateral(4,2), params.A_lateral(4,4)];
    B_2state = [params.B_lateral(2,2); params.B_lateral(4,2)];
end

function results = initialize_controller_results()
    results = struct('synthesis_success', false, 'method', '', 'K_beta', NaN, 'K_r', NaN, ...
        'h2_norm', NaN, 'gamma_inf', NaN, 'closed_loop_poles', [], ...
        'optimization_iterations', NaN, 'final_cost', NaN, 'cost_history', []);
end

function validate_parameters(params)
    assert(all(real(eig(params.A_lateral)) < 0.01), 'Open loop aircraft dynamics are unstable');
end

function [temp_K, pressure_Pa, density_kgm3] = calculate_atmospheric_properties(altitude_m)
    T0 = 288.15; L = 0.0065; g = 9.80665; R = 287.053;
    temp_K = T0 - L * altitude_m;

```



```

        pressure_Pa = 101325 * (temp_K / T0)^(g / (R * L));
        density_kgm3 = pressure_Pa / (R * temp_K);
    end

function display_flight_conditions(params)
    fprintf('Flight Conditions:\n');
    fprintf(' Altitude:      %d ft (%.1f m)\n', params.altitude_ft, params.altitude_m);
    fprintf(' Airspeed:      %d KTAS (%.1f m/s)\n', params.airspeed_knots, params.airspeed_ms);
    fprintf(' Turbulence:      %s\n', params.turbulence_intensity);
end

function x_next = rk4_integration(x_current, state_derivatives, dt)
    k1 = dt * state_derivatives;
    k2 = dt * state_derivatives;
    k3 = dt * state_derivatives;
    k4 = dt * state_derivatives;
    x_next = x_current + (k1 + 2*k2 + 2*k3 + k4)/6;
end

function aileron_cmd = generate_aileron_command(time, ~)
    if time >= 2 && time < 4, aileron_cmd = 12.5 * pi/180;
    elseif time >= 5 && time < 7, aileron_cmd = -12.5 * pi/180;
    else, aileron_cmd = 0; end
end

function J = compute_comfort_cost_function(states, controls, params)
    beta = states(2, :);
    r = states(4, :);
    phi = states(5, :);
    delta_r = controls(2, :);
    beta_dot = states(3, :);
    lat_accel_ms2 = params.airspeed_ms * (r - params.GRAVITY/params.airspeed_ms .* sin(phi)) +
    params.airspeed_ms .* beta_dot;
    lat_accel_g = lat_accel_ms2 / params.GRAVITY;
    dt = params.time_step;
    rudder_rate = [0, diff(delta_r)/dt];
    w1 = 10.0; w2 = 15.0; w3 = 5.0; w4 = 3.0;
    J = w1 * rms(beta/params.DEG_TO_RAD)^2 + ...
        w2 * rms(lat_accel_g)^2 + ...
        w3 * rms(delta_r/params.DEG_TO_RAD)^2 + ...
        w4 * rms(rudder_rate/params.DEG_TO_RAD)^2;
end

function gradient = compute_numerical_gradient(params, K_current, gust_series, J_current)
    epsilon = [0.01; 0.01];
    gradient = zeros(2, 1);
    for i = 1:2
        K_plus = K_current;
        K_plus(i) = K_plus(i) + epsilon(i);
        K_plus(1) = max(0, min(12, K_plus(1)));
        K_plus(2) = max(0, min(6, K_plus(2)));
        temp_controller = struct('synthesis_success', true, 'K_beta', K_plus(1), 'K_r', K_plus(2));
        [~, states_plus, controls_plus] = simulate_2state_optimized_response(params, temp_controller,
        gust_series);
        J_plus = compute_comfort_cost_function(states_plus, controls_plus, params);
        gradient(i) = (J_plus - J_current) / epsilon(i);
    end
end

```

```

function H_inv_new = update_inverse_hessian(H_inv_old, s, y)
    rho = 1 / (y' * s);
    if abs(rho) > 1e6 || ~isfinite(rho)
        H_inv_new = H_inv_old;
        return;
    end
    I = eye(size(H_inv_old));
    H_inv_new = (I - rho * s * y') * H_inv_old * (I - rho * y * s') + rho * s * s';
    [V, D] = eig(H_inv_new);
    D = diag(max(diag(D), 1e-6));
    H_inv_new = V * D * V';
End

```

CN235-220 Flight Test Report